UNIVERSIDAD ALFONSO X EL SABIO

BUSINESS & TECH

GRADO EN INFORMÁTICA

(GIF)



TRABAJO FIN DE GRADO

DISEÑO DE UN SISTEMA DE ASISTENCIA AL ANÁLISIS TÁCTICO DEPORTIVO BASADO EN PROCESAMIENTO DE IMAGEN

HUGO SÁNCHEZ-GALLEGO GÁLVEZ

26/06/2026

UNIVERSIDAD ALFONSO X EL SABIO

BUSINESS & TECH

GRADO EN INFORMÁTICA - GIF

DISEÑO DE UN SISTEMA DE ASISTENCIA AL ANÁLISIS TÁCTICO DEPORTIVO BASADO EN PROCESAMIENTO DE IMAGEN

ALUMNO: HUGO SÁNCHEZ-GALLEGO GÁLVEZ

N.P.: 145975

DIRECTOR DEL TRABAJO:

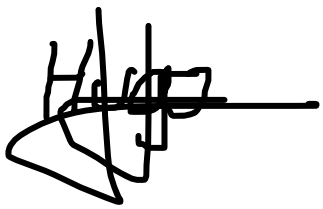
TUTOR ACADÉMICO: BEATRIZ MAGÁN PINTO

FECHA DE PRESENTACIÓN: 26/06/2026

BREVE DESCRIPCIÓN:

Sistema de visión por computador que a partir de un único vídeo de retransmisión y sobre hardware de consumo, detecta a los jugadores, al árbitro y al balón, mantiene su identidad con seguimiento multiobjeto, clasifica a cada jugador por equipo según el color de la camiseta y calibra el campo para proyectar las posiciones sobre un plano en dos dimensiones. Sobre esas posiciones calcula métricas tácticas como el centroide, la amplitud del bloque, los mapas de calor o la posesión, e integra todo el proceso en una tubería que procesa el vídeo completo y vuelca su salida en un fichero JSON reutilizable, con los modelos publicados en abierto.

FIRMA DEL ALUMNO



UAX Universidad Alfonso X el Sabio

Resumen

Este trabajo presenta un sistema de análisis táctico de fútbol que parte de un único vídeo de retransmisión y funciona sobre hardware de consumo. El sistema detecta a los jugadores, al árbitro y al balón con un modelo YOLOv8 entrenado sobre imágenes de partidos reales, mantiene la identidad de cada jugador con el seguidor BoT-SORT y un cosido posterior de las trayectorias partidas, y asigna cada futbolista a su equipo agrupando por el color de la camiseta sin etiquetas previas. En paralelo, la calibración del terreno de juego con PnLCalib proporciona la homografía que relaciona la imagen con las coordenadas reales del campo. Las dos vías confluyen en una proyección que sitúa a cada jugador sobre un plano en dos dimensiones, sobre el que un motor táctico calcula medidas como el centroide del equipo, la amplitud del bloque, los mapas de calor, la posesión y la orientación del ataque. Todo ello queda integrado en una tubería que procesa un vídeo completo modificando un solo parámetro y entrega su salida en un fichero JSON reutilizable. La evaluación sobre secuencias públicas de SoccerNet-Tracking confirma una detección de personas sólida, un seguimiento que conserva bien la identidad y una clasificación de equipos en torno al noventa y dos por ciento, con el balón como principal punto débil. Los modelos entrenados y el código se publican en abierto.

Abstract

This work presents a football tactical analysis system that starts from a single broadcast video and runs on consumer hardware. The system detects players, the referee and the ball with a YOLOv8 model trained on real match footage, keeps each player's identity over time with the BoT-SORT tracker followed by a stitching step that repairs broken trajectories, and assigns every player to a team by clustering shirt colours without prior labels. In parallel, field calibration with PnLCalib provides the homography that maps the image onto real pitch coordinates. Both paths meet in a projection that places each player on a two-dimensional plan, on which a tactical engine computes measures such as the team centroid, block width, heat maps, possession and attacking direction. Everything is integrated into a pipeline that processes a full video by changing a single parameter and writes its output to a reusable JSON file. Evaluation on public SoccerNet-Tracking sequences confirms solid person detection, tracking that preserves identity well and team classification of around ninety-two per cent, with the ball remaining the main weak point. The trained models and the source code are released openly.

UAX Universidad Alfonso X el Sabio

Índice general

Resumen **1**

Abstract **2**

1. Introducción **6**
1.1. Contexto 6
1.2. Justificación del estudio 7
1.3. Estructura del documento 8

2. Antecedentes y estado del arte **9**
2.1. El análisis táctico y la necesidad del dato posicional 9
2.1.1. Del registro manual al dato espacial 9
2.1.2. Qué se mide con las posiciones 10
2.1.3. Coste y motivación 11
2.2. De los píxeles a la representación: cómo una máquina aprende a ver . 11
2.2.1. Procesamiento clásico 12
2.2.2. La irrupción de las CNN 12
2.2.3. Atención y transformers 13
2.2.4. La detección 13
2.3. Mantener la identidad en el tiempo: el seguimiento 14
2.3.1. El núcleo matemático 14
2.3.2. La evolución frente a la oclusión 15
2.3.3. El reto del deporte 17
2.4. Interpretar el color para distinguir equipos 17
2.4.1. Por qué el color engaña 17

2.4.2. Agrupar sin etiquetas . . . 18
2.4.3. Cuándo se rompe . . . 19
2.5. Anclar la imagen al espacio: pose y puntos clave . . . 19
2.5.1. Localizar estructuras . . . 19
2.5.2. Enfoques . . . 20
2.5.3. Datasets y frontera . . . 21
2.6. Del plano imagen al plano del campo: geometría proyectiva . . . 22
2.6.1. Modelo pinhole y homografía . . . 22
2.6.2. Resolver con robustez . . . 24
2.6.3. Estabilidad y frontera . . . 26
2.7. Integración y evaluación . . . 27
2.7.1. La tarea como problema formal . . . 27
2.7.2. Ensamblaje y modelos unificados . . . 28
2.7.3. Referencias comerciales y el hueco . . . 29

3. Objetivos 30

4. Metodología 32
4.1. Metodología de desarrollo . . . 32
4.1.1. Justificación de la metodología . . . 32
4.1.2. Las seis fases aplicadas al proyecto . . . 32
4.1.3. Carácter iterativo y gestión de resultados negativos . . . 33
4.2. Entorno de desarrollo . . . 34
4.2.1. Hardware . . . 34
4.2.2. Lenguaje y librerías . . . 34
4.2.3. Herramientas de soporte . . . 34
4.3. Conjuntos de datos . . . 35
4.3.1. Vídeos de la Bundesliga . . . 35
4.3.2. SoccerNet-Calibration . . . 35
4.3.3. SoccerNet-Tracking . . . 36
4.4. Arquitectura del sistema . . . 36
4.4.1. Flujo del sistema . . . 36
4.4.2. Interfaz de datos entre módulos . . . 36
4.5. Metodología de evaluación . . . 37
4.5.1. Métricas de detección y seguimiento . . . 37
4.5.2. Protocolo de evaluación . . . 38
4.5.3. Evaluación de la calibración . . . 38

5. Desarrollo y resultados 40
5.1. Detección de objetos . . . 40

5.1.1. Construcción iterativa del dataset 40
5.1.2. Evolución de los modelos y entrenamiento 42
5.1.3. Resultados y selección del modelo 43
5.2. Seguimiento y clasificación de equipos 45
5.2.1. Seguimiento de jugadores 46
5.2.2. Post-procesado de los tracklets 47
5.2.3. Clasificación de equipos y roles 48
5.3. Calibración del campo y proyección a la pizarra 50
5.3.1. Modelo de keypoints base 51
5.3.2. Un fine-tuning en paralelo 51
5.3.3. De los keypoints a la homografía 52
5.3.4. La cobertura, el punto débil de mi modelo 53
5.3.5. El cambio a PnLCalib 54
5.3.6. Proyección a la pizarra 54
5.4. Motor táctico 55
5.4.1. Métricas de bloque 55
5.4.2. Mapas de calor y posesión 56
5.4.3. Formación y dirección de ataque 57
5.4.4. Doble visualización 58
5.5. Pipeline integrado 59
5.6. Evaluación cuantitativa 61
5.6.1. Detección, seguimiento y clasificación 61
5.6.2. Efecto del post-procesado 62

6. Conclusiones **63**
6.1. Líneas futuras de trabajo 65

7. Abreviaturas **67**

CAPÍTULO 1

Introducción

1.1 › Contexto

El fútbol mueve hoy unas cifras que han terminado por profesionalizar cada rincón de su preparación. Lo que durante mucho tiempo se decidía con el ojo de un técnico y una libreta se apoya cada vez más en datos, y de todos ellos los que más han crecido son los que dicen dónde está cada jugador en cada momento. Conocer la posición de los veintidós futbolistas a lo largo del partido permite medir cosas que antes solo se intuían, desde la altura a la que un equipo aprieta hasta el espacio que concede cuando se lanza al ataque.

Capturar esa posición nunca ha sido gratis. Durante años la única manera fiable de obtenerla pasaba por instalar sistemas ópticos de varias cámaras repartidas por el estadio o por colgar receptores GPS de los jugadores en los entrenamientos. Una y otra vía devuelven medidas muy precisas, pero cargan con un coste de instalación y de servicio que rara vez baja de la élite. Quedan fuera del alcance de la inmensa mayoría de clubes, de las categorías inferiores y de cualquiera que quiera estudiar el juego sin un presupuesto detrás.

En paralelo, la visión por computador ha cambiado de raíz en poco más de una década. El paso de las técnicas artesanales a las redes neuronales convolucionales, y después la aparición de arquitecturas cada vez más finas, han llevado la detección de objetos sobre imagen a un punto que hace no tanto parecía reservado a la percepción humana. Hoy un modelo entrenado localiza a decenas de personas en un fotograma y lo hace en tiempo casi real sobre una tarjeta gráfica de las que caben en un portátil.

El deporte ha sido uno de los terrenos donde ese avance ha calado antes, porque hay mucho vídeo grabado y mucho interés en sacarle partido.

De ese cruce surge una fuente de datos que estaba ahí desde siempre y que solo ahora empieza a ser explotable, la propia retransmisión televisiva. Cualquier partido emitido es, en la práctica, un conjunto de imágenes esperando a ser leído, sin más requisito que disponer de la grabación. La contrapartida es que esa señal procede de una sola cámara que persigue el balón y cierra el plano sobre la jugada, con buena parte del campo fuera del encuadre a cada instante, lo que convierte la extracción de posiciones fiables en un problema bastante más denso que leerlas de una instalación pensada para ese fin.

El interés por recorrer ese camino se ha reforzado por los nuevos descubrimientos. Han aparecido modelos capaces de proponer y evaluar jugadas de estrategia partiendo únicamente de las coordenadas de los jugadores y cuanto más capaces se vuelven esas herramientas más sentido tiene poder alimentarlas desde una fuente tan extendida como la televisión en vez de depender de capturas que muy pocos pueden costear. El entorno en el que se mueve este trabajo es ese, el de una tecnología que ha madurado lo justo para intentar arrancar de un vídeo de emisión lo que hasta hace poco exigía una infraestructura cara.

1.2 › Justificación del estudio

Si los sistemas que producen datos posicionales de calidad están reservados a quien puede pagarlos, conviene preguntarse hasta dónde se llega con lo que sí está al alcance de cualquiera. Esa es la pregunta que sostiene el trabajo. Un solo vídeo de retransmisión y una máquina de consumo son el punto de partida más humilde posible para un problema que la industria resuelve con despliegues de seis cifras, y comprobar cuánto análisis táctico se puede recuperar desde ahí es una aportación que va más allá del ejercicio académico.

La relevancia no se centra solo en el acceso aunque sea lo primero que salta a la vista. Acercar la extracción de posiciones a quien no tiene detrás la estructura de un club grande abre la puerta a que entrenadores de base, periodistas o aficionados con conocimientos técnicos trabajen con un material que hasta ahora les era ajeno. Pero hay algo más que tiene que ver con la integración. El proyecto no inventa una pieza aislada sino que encadena detección, seguimiento, clasificación de equipos, calibración del campo y cálculo de métricas en una sola tubería que arranca de un fichero de vídeo y termina en una representación táctica. Sostener ese encadenamiento entero sobre hardware limitado es donde se concentra buena parte del reto de ingeniería.

A todo esto se suma la voluntad de dejar el resultado abierto. Los modelos entrenados y el código quedan publicados bajo una licencia permisiva de modo

que cualquiera pueda reutilizarlos, contrastarlos o construir encima. En un campo donde las soluciones más potentes son también las más cerradas poner a disposición de todos una alternativa modesta pero funcional tiene un valor que no se mide solo en métricas.

1.3 › Estructura del documento

La memoria sigue el recorrido natural del proyecto, de lo general a lo concreto. Tras esta introducción, el capítulo de antecedentes y estado del arte repasa de dónde viene el análisis posicional en el fútbol y qué herramientas existen para cada una de las piezas que el sistema necesita, desde la detección de objetos hasta la geometría que relaciona la imagen con el campo. Sirve para situar el trabajo dentro de lo que ya se ha hecho y para entender las elecciones técnicas que vienen después.

El capítulo de objetivos fija qué se quiere conseguir y con qué límites, y es el que marca la vara con la que se miden los resultados al final. A continuación, la metodología explica cómo se ha trabajado. Describe el proceso de desarrollo guiado por CRISP-DM, el entorno de hardware y software sobre el que se ha construido todo, los conjuntos de datos empleados, la arquitectura del sistema y el protocolo con el que se evalúa.

El grueso técnico está en el capítulo de desarrollo y resultados, que recorre módulo a módulo la construcción del sistema. La detección de objetos, el seguimiento y la clasificación de equipos, la calibración del campo y la proyección, el motor táctico que calcula las métricas y el pipeline que une todas las piezas, cada uno con las decisiones que se fueron tomando y con lo que se obtuvo al ponerlo a prueba. Cierra ese capítulo la evaluación cuantitativa sobre el conjunto de test.

Las conclusiones recogen en qué medida se han cubierto los objetivos y abren las líneas de trabajo que quedan pendientes. Después un capítulo de abreviaturas reúne la terminología técnica y los acrónimos que aparecen a lo largo del texto. La memoria termina con los índices de figuras y de tablas y con la bibliografía.

CAPÍTULO 2

Antecedentes y estado del arte

2.1 › El análisis táctico y la necesidad del dato posicional

2.1.1 › Del registro manual al dato espacial

Durante décadas el análisis del juego se hizo a ojo. Un técnico veía el partido y apuntaba a mano lo que iba pasando y con esas notas reconstruía después lo sucedido. El método capturaba bien los eventos. Lo que se le escapaba era casi todo lo que ocurría entre uno y otro, el movimiento de los jugadores lejos del balón o la forma en que el equipo se estiraba y se encogía según el momento. Con la profesionalización del fútbol y la reducción de costes la atención se desplazó hacia ese material que antes se perdía y la posición de cada futbolista a lo largo del tiempo empezó a tratarse como un dato en sí mismo [1].

Capturar esa posición exige instrumentar el partido de alguna manera, y por ahí han pasado tecnologías muy distintas. Las primeras en extenderse fueron las que lleva encima el propio jugador. Los sistemas EPTS y los receptores GPS integrados en una prenda registran la trayectoria con una precisión razonable y se emplean sobre todo en los entrenamientos y en las competiciones que permiten llevarlos puestos. De esas trazas se obtienen los metros recorridos, las velocidades alcanzadas, el número de esfuerzos de alta intensidad y todo un conjunto de indicadores de carga que hoy revisa cualquier preparador físico [2].

Hay una segunda familia que no toca al jugador. El seguimiento óptico multicá-

mara reparte varias cámaras calibradas por el estadio y reconstruye desde la imagen la posición de todo el que pisa el campo, sin un solo sensor encima de nadie [3]. El problema es que necesita una instalación fija y cara, que solo existe en recintos preparados para alojarla.

La vía que ha llegado más tarde busca ese mismo dato en la señal que se emite por televisión. Aquí no hay despliegue de cámaras. Hay una sola, la del realizador, que va detrás del balón y cierra el plano sobre la jugada, con gran parte del terreno fuera del encuadre en todo momento. Es la fuente más accesible que existe y a la vez la más complicada de explotar, y esa misma dificultad ha motivado bancos de pruebas creados a propósito para medir el seguimiento de jugadores sobre vídeo de emisión [4].

2.1.2 › Qué se mide con las posiciones

Cuando se dispone de las coordenadas de los jugadores se puede empezar a describir el juego colectivo con números y es ahí donde el dato posicional demuestra su utilidad real para el análisis [5].

La medida más elemental es el centroide del equipo, que no es más que la posición media de sus once jugadores sobre el campo. Seguir su recorrido a lo largo del partido resulta muy útil. Si el centroide se adelanta, el equipo ha subido sus líneas para presionar arriba. Si retrocede, se ha replegado cerca de su propia área. A esa medida se le añade casi siempre el grado de concentración del conjunto, porque dos equipos con idéntico centroide pueden estar uno muy junto y otro muy abierto. El índice de dispersión es el que captura esa diferencia tanto a lo largo como a lo ancho [6].

La superficie que cubre un equipo aporta una lectura distinta. Se obtiene trazando el polígono más pequeño que encierra a todos los jugadores, conocido como envolvente convexa o convex hull. Ese área se ensancha cuando el equipo se abre para atacar y se estrecha cuando se encierra en bloque y observada a lo largo de los minutos deja ver cómo respira el equipo entre las distintas fases del juego [5].

El control del espacio da un paso más, porque ya no atiende a dónde están los jugadores sino a qué porción de campo gobierna cada uno. La herramienta habitual es el diagrama de Voronoi, que divide el terreno en regiones y adjudica cada punto al jugador que lo tiene más próximo. Al sumar las regiones de un mismo equipo se obtiene una estimación del espacio que controla, lo que permite hablar de superioridad posicional en zonas determinadas sin reducirlo todo a quién está más cerca del balón [5].

De por sí solos, estos números no significan gran cosa. Adquieren sentido al leerlos teniendo el contexto del juego. Que un bloque se mantenga alto y compacto delata una voluntad de presionar en campo contrario, mientras que una superficie que se dilata y se contrae va marcando el tránsito entre defender y atacar. El reparto

del espacio que ofrece el Voronoi enlaza, por su parte, con nociones de ocupación y de generación de huecos que los entrenadores manejan desde siempre de manera intuitiva, aunque rara vez les hayan puesto una cifra delante [1].

2.1.3 › Coste y motivación

La precisión del dato y lo que cuesta obtenerlo van bastante de la mano. Los sistemas ópticos dedicados, con nombres comerciales como TRACAB o Second Spectrum, devuelven posiciones muy precisas de los veintidós jugadores y del balón. El precio de esa precisión es una instalación multicámara permanente y un servicio contratado, algo reservado a clubes y competiciones con presupuesto para asumirlo [3].

En el extremo opuesto, el vídeo emitido por televisión es la fuente más universal que hay. Cualquier partido retransmitido es ya, en la práctica, un conjunto de datos a la espera de ser aprovechado, sin más requisito que disponer de la grabación. Esa comodidad, sin embargo, se paga por otro lado. La imagen procede de una única cámara que persigue la jugada y cierra el plano, con una parte considerable del campo quedando fuera en cada instante, y reconstruir posiciones fiables a partir de ahí es mucho más difícil que leerlas de un sistema montado para ese fin. Cuánto de difícil es exactamente eso es lo que han venido a cuantificar los bancos de pruebas levantados sobre emisión real [4].

El interés por recorrer este camino no se reduce al acceso. El análisis táctico apoyado en aprendizaje automático ha dado pasos notables en los últimos años. Existen ya modelos capaces de proponer y evaluar jugadas de estrategia partiendo únicamente de las posiciones de los futbolistas [7]. Cuanto más capaces son esas herramientas, más sentido tiene poder alimentarlas con datos espaciales extraídos de una fuente tan extendida como la televisión en lugar de depender de capturas que muy pocos pueden costear.

En el cruce de esas dos realidades aparece el hueco. Los sistemas profesionales producen un dato riquísimo que casi nadie puede pagar. La televisión ofrece una fuente barata y omnipresente de la que todavía cuesta extraer ese mismo dato con garantías. Entre una y otra se abre camino la vía de una sola cámara sobre vídeo de retransmisión, que es la que aspira a acercar el análisis posicional a quien no tiene detrás la infraestructura de un club grande.

2.2 › De los píxeles a la representación: cómo una máquina aprende a ver

2.2.1 › Procesamiento clásico

Para una cámara una imagen no es una escena con jugadores y un balón. Es un conjunto de números, un valor de intensidad por píxel y por canal de color. Convertir esas cifras en algo parecido a información útil fue el primer gran problema de la visión por computador, y durante mucho tiempo se abordó con operaciones diseñadas a mano.

La pieza elemental de casi todas ellas es la convolución. Consiste en deslizar una pequeña matriz de pesos, llamada núcleo o filtro, sobre la imagen, multiplicando y sumando en cada posición para obtener un valor nuevo. Cambiando los pesos cambia el efecto. Un filtro gaussiano promedia el vecindario de cada píxel y suaviza la imagen, lo que ayuda a atenuar el ruido. Un filtro de Sobel responde a las variaciones bruscas de intensidad y resalta los bordes que son las fronteras donde cambia el contenido [8].

Detectar bordes y suavizar es poco todavía. El paso siguiente fue construir descriptores, que son vectores que resumen la apariencia local de una región para poder compararla o clasificarla. De aquí podemos destacar 2: SIFT localiza puntos característicos estables frente a cambios de escala y rotación y los describe de forma que se puedan reconocer en otra imagen [8]. HOG divide la imagen en celdas, calcula en cada una un histograma de orientaciones del gradiente y concatena esos histogramas en un descriptor que captura la forma a partir de la distribución de sus bordes [9]. Sobre HOG se levantó el detector de peatones que durante años fue la referencia.

El problema de todo este sistema asoma en cuanto las condiciones cambian. Un descriptor afinado a mano funciona mientras la iluminación, el fondo y el tamaño del objeto se parezcan a aquello para lo que se pensó, y se deteriora en cuanto algo se sale del guion. En el fútbol eso pasa sin parar. La luz del estadio cambia entre el sol y la sombra, los jugadores se ven enormes cerca de la cámara y minúsculos al fondo, se tapan unos a otros cuando se agrupan y, encima, casi todo un equipo viste igual. Cada mejora obligaba a rediseñar las características casi desde cero, y ese trabajo manual no escalaba.

2.2.2 › La irrupción de las CNN

El cambio consistió en dejar de decirle a la máquina qué mirar. En vez de diseñar los filtros y los descriptores se dejó que una red los aprendiera ella sola a partir de ejemplos.

Una red neuronal convolucional apila muchas capas de convoluciones cuyos pesos no se fijan a mano sino que se ajustan durante el entrenamiento. Lo llamativo es lo que aprende. Las primeras capas terminan respondiendo a bordes y manchas de color, no muy lejos de los filtros clásicos. Las intermedias combinan esas respuestas en motivos y partes, y las últimas reaccionan ante objetos completos. Nadie programa

esa jerarquía que va de lo simple a lo complejo. Surge sola del proceso de ajuste y resultó ser mucho más robusta que cualquier característica escrita a mano.

El punto de inflexión llegó en 2012. Una red convolucional profunda ganó por un margen amplísimo el principal concurso de clasificación de imágenes, apoyado en ImageNet, un repositorio de millones de imágenes etiquetadas, y dejó muy atrás a los métodos de características artesanales [10]. Dos cosas lo hicieron posible: por un lado, el volumen de datos disponible para entrenar. Por otro, la potencia de las tarjetas gráficas, que permitió ajustar redes de millones de parámetros en un tiempo razonable. Desde entonces el aprendizaje profundo pasó a ser el enfoque dominante en visión.

2.2.3 › Atención y transformers

La convolución mira siempre cerca. Cada neurona de una capa convolucional combina información de una vecindad pequeña, y relacionar dos zonas alejadas de la imagen exige apilar muchas capas. Esa limitación de alcance abrió la puerta a otra idea venida del procesamiento del lenguaje.

Esa idea es la atención. En un mecanismo de atención cada elemento de la entrada pondera la relevancia de todos los demás y arma su representación a partir de esa mezcla sin importar lo lejos que estén entre sí. El trabajo que la dio a conocer demostró que una arquitectura basada por entero en atención (el transformador) bastaba para tareas de lenguaje que hasta entonces dependían de redes recurrentes [11].

Trasladarlo a la imagen pedía un pequeño rodeo. El Vision Transformer trocea la imagen en parches cuadrados, los ordena como las palabras de una frase y deja que la atención los relacione a todos entre sí, prescindiendo casi del todo de la convolución [12]. Los transformadores no han desterrado a las redes convolucionales ya que siguen siendo la opción práctica en muchos escenarios. Lo que han hecho es ampliar el repertorio de arquitecturas y empujar hacia diseños más homogéneos.

2.2.4 › La detección

Clasificar una imagen entera responde a una pregunta sencilla: ¿qué hay en ella? La detección de objetos es más ambiciosa porque pide decir qué hay y además dónde, dibujando una caja alrededor de cada elemento. Sobre esa capacidad se apoya cualquier sistema que aspire a situar a los jugadores en el campo.

Las soluciones modernas se reparten en dos familias separadas entre precisión y velocidad. La primera trabaja en dos etapas. Una parte de la red propone regiones donde podría haber algo y otra examina cada propuesta para clasificarla y ajustar su caja. Faster R-CNN es el exponente más conocido de esta línea e integró la generación de propuestas dentro de la propia red para acelerarla, aunque revisar

cientos de candidatos por imagen mantiene un coste que lo penaliza en tiempo real [13].

La segunda familia suprime esa etapa intermedia. Plantea la detección como una única regresión que, en una sola pasada de la red sobre toda la imagen, devuelve directamente las cajas y sus clases. Esa es la propuesta de YOLO desde el principio para correr muy rápido [14]. La velocidad le costó al principio algo de precisión con los objetos pequeños pero la distancia se ha ido acortando versión tras versión.

Existe además una vía más reciente que lleva la atención a la detección. DETR reformula el problema como una predicción de conjuntos en la que la red emite de una vez el conjunto de objetos presentes sin proponer regiones intermedias ni necesitar el filtrado posterior de cajas solapadas que arrastraban los detectores anteriores [15].

Dentro de la familia de una sola pasada la línea de YOLO ha seguido evolucionando con cambios en su arquitectura y en su entrenamiento. Sus versiones recientes han abandonado las cajas de referencia predefinidas y predicen la caja directamente, un enfoque llamado anchor-free que simplifica la configuración y maneja mejor los objetos de tamaños muy dispares. Han reorganizado el extractor de características en torno a un módulo, el C2f, que combina información de distinto nivel para afinar la detección y han adoptado una pérdida de tipo CIoU para la regresión de la caja, que no atiende solo al solapamiento sino también a la distancia entre centros y a la relación de aspecto [16].

2.3 › Mantener la identidad en el tiempo: el seguimiento

2.3.1 › El núcleo matemático

El esquema que domina la disciplina se conoce como seguimiento por detección. Un detector localiza a los objetos en cada cuadro y un módulo de seguimiento enlaza esas localizaciones con las trayectorias que venía arrastrando de cuadros anteriores. La dificultad se reparte en dos tareas distintas. La primera consiste en anticipar dónde estará cada objeto cuando llegue la imagen siguiente. La segunda, en resolver qué detección recién aparecida pertenece a cada trayectoria ya abierta.

La predicción se aborda casi siempre con el filtro de Kalman, un estimador que modela el estado de cada objeto y lo proyecta hacia delante en el tiempo [17]. En la formulación que popularizó SORT ese estado reúne la posición del centro de la caja, su escala y su relación de aspecto acompañadas de sus velocidades, siempre y cuando el movimiento entre cuadros consecutivos sea aproximadamente uniforme. El filtro funciona en dos fases que se repiten en cada imagen. Durante la predicción adelanta el estado al instante siguiente aplicando el modelo de movimiento y arras-

tra consigo una estimación de la incertidumbre asociada. Durante la actualización corrige esa predicción con la detección que acaba de observar y reduce la incertidumbre, ponderando ambas fuentes de información en función de lo fiable que sea cada una. Ese peso (la ganancia de Kalman) sube cuando la medida es de fiar y baja cuando conviene apoyarse en el modelo. La consecuencia práctica importa mucho en este problema, porque si una oclusión deja a un jugador sin detección durante unos cuantos cuadros el filtro sigue propagando una posición esperada cada vez más incierta, lo que da margen para reengancharlo cuando vuelve a ser visible.

Con las posiciones predichas disponibles queda emparejarlas con las detecciones reales del cuadro. Se trata de un problema de asignación, el de repartir trayectorias y observaciones de modo que cada predicción se case con la detección que mejor le corresponde y ninguna quede usada dos veces. La afinidad entre una caja predicha y una detección se mide habitualmente por su solapamiento (el índice IoU), que se transforma en un coste tomando su complementario de manera que a mayor solape menor coste de unirlas. El reparto que minimiza el coste total se calcula con el algoritmo húngaro, un procedimiento clásico de investigación operativa que encuentra la asignación óptima en tiempo polinómico sin tener que probar todas las combinaciones [18]. Antes de aceptar un emparejamiento se aplica un umbral mínimo de solape que descarta las uniones forzadas, las de cajas que apenas se rozan y que casi con seguridad pertenecen a objetos distintos.

Queda un tercer factor que las descripciones a veces dan por supuesto: la gestión del ciclo de vida de las trayectorias. El seguidor decide cuándo nace una trayectoria, normalmente tras confirmar una detección durante varios cuadros seguidos para no crear pistas a partir de un falso positivo aislado, y decide también cuándo se da por muerta, cuando lleva demasiado tiempo sin recibir ninguna detección que la sostenga. Ese recuento de cuadros sin observación es el que fija cuánta oclusión tolera el sistema antes de rendirse y soltar la identidad.

2.3.2 › La evolución frente a la oclusión

Sobre ese núcleo de Kalman y asignación húngara se ha construido toda una familia de seguidores. La evolución se entiende bien si se mira qué corrige cada propuesta respecto a la que la precede.

El punto de partida es SORT [19]. Combina el filtro de Kalman con la asociación húngara sobre el IoU y prácticamente nada más, sin ningún modelo del aspecto de los objetos. Esa sencillez es a la vez su virtud y su límite. Por un lado lo hace extremadamente rápido y apto para el tiempo real con un detector decente por delante. Por otro lado lo hace vulnerable al movimiento como única pista, de modo que cuando dos jugadores se cruzan o uno desaparece tras otro durante un tramo largo la predicción envejecida solapa poco con la detección que reaparece y el seguidor

termina asignando un identificador nuevo. Esos cambios de identidad, abundantes en cuanto la escena se complica, son el problema que arrastra.

DeepSORT se diseñó para corregir esa debilidad [20]. A la información de movimiento le suma un descriptor de apariencia, que consiste en un vector que resume el aspecto visual de cada objeto y que genera una red convolucional entrenada previamente para reidentificación de personas. Con él la asociación deja de depender solo de la posición prevista e incorpora una medida de parecido visual. Esto permite recuperar la identidad de un jugador que reaparece tras una oclusión aunque haya cambiado de sitio. La decisión combina dos distancias, una de movimiento calculada sobre el estado del filtro y otra de apariencia entre descriptores, y se ejecuta mediante una cascada que atiende primero a las trayectorias confirmadas más recientemente y deja para el final las que llevan varios cuadros sin verse. El precio es el cómputo añadido de extraer un descriptor por cada detección en cada cuadro.

El giro de ByteTrack fue distinto [21]. En vez de refinar la apariencia se fijó en algo que el resto descartaba de entrada: las detecciones de confianza baja. Cuando un jugador queda medio tapado el detector le asigna una puntuación pobre que el umbral habitual eliminaría por considerarla ruido, pese a que detrás hay un objeto perfectamente real. La idea consiste en asociar en dos rondas y no tirar ninguna caja. La primera ronda empareja las detecciones de puntuación alta con las trayectorias activas como haría cualquier seguidor. La segunda intenta rescatar las detecciones de puntuación baja enfrentándolas a las trayectorias que se han quedado sin pareja en la ronda anterior, que son justamente las de los objetos que la oclusión ha degradado. Con esa maniobra y apoyándose casi por completo en el movimiento sin el sobrecoste de la apariencia, retiene buena parte de las identidades que se perderían en las aglomeraciones.

BoT-SORT figura entre los seguidores de esta familia y encaja especialmente bien con el vídeo de retransmisión [22]. Su contribución más valiosa para el fútbol televisado es la compensación del movimiento de la cámara. La realización de un partido persigue la jugada con desplazamientos laterales y zooms de imagen, y ese movimiento global hace que toda la escena se mueva entre cuadros. Esto desbarata un filtro de Kalman que supone la cámara quieta y atribuye a los jugadores un desplazamiento que en realidad es de la cámara. BoT-SORT estima ese movimiento global rastreando puntos característicos del fondo de un cuadro al siguiente y lo descuenta de la predicción antes de asociar, con lo que separa el movimiento propio de cada jugador del inducido por la realización. Además, ajusta la parametrización del estado de Kalman para estimar directamente el ancho y el alto de la caja en lugar de su área y su proporción y vuelve a integrar la apariencia junto al solape geométrico recogiendo lo mejor de las dos ramas anteriores.

2.3.3 › El reto del deporte

Casi toda esta maquinaria nació y se midió con peatones grabados por cámaras de seguridad, un escenario en el que cada persona viste distinto y la apariencia separa a unas de otras con comodidad. El fútbol invalida ese punto de partida. La pieza que sostiene a DeepSORT y a BoT-SORT (el descriptor de apariencia) pierde casi todo su poder discriminante cuando los once jugadores de un equipo llevan la misma camiseta y coinciden amontonados en un saque de esquina o en una barrera. A la uniformidad de aspecto se suman la velocidad de los movimientos, las oclusiones mutuas que se encadenan en cuanto hay aglomeración y el desenfoque del movimiento rápido, que difumina unos rasgos que ya de por sí son poco distintivos. En esas condiciones los seguidores generalistas disparan el número de cambios de identidad que es la métrica donde más se sufre [3].

Para estudiar el problema sin sacarlo de su contexto se han publicado conjuntos de datos pensados para el deporte. SportsMOT reúne secuencias de varias disciplinas que comparten el movimiento veloz y la apariencia homogénea entre participantes y se emplea para comprobar cómo aguantan los seguidores cuando esas dos dificultades aparecen juntas [23]. Centrado ya en el fútbol, SoccerNet añade a su colección una tarea de seguimiento sobre vídeo de retransmisión que traslada al banco de pruebas las condiciones reales de la emisión televisiva con sus cortes y sus cambios de plano [4].

Actualmente se está experimentando con seguidores de extremo a extremo levantados sobre el transformador que se presentó en el apartado 2.2.3 [11]. La idea rompe con la tubería clásica. En lugar de encadenar como módulos independientes un detector, un filtro de Kalman y una asociación húngara, estos modelos mantienen a lo largo de los cuadros un conjunto de consultas que representan a cada objeto en seguimiento y dejan que la propia red aprenda a la vez a detectar y a enlazar en el tiempo [24]. Es una vía aún en desarrollo que necesita de muchos más datos y de cómputo que la aproximación modular y que todavía está lejos de desplazarla en la práctica, pero marca la dirección hacia la que avanza el seguimiento multiobjeto.

2.4 › Interpretar el color para distinguir equipos

2.4.1 › Por qué el color engaña

Una vez que el sistema sigue a cada jugador a lo largo del vídeo falta por resolver a qué equipo pertenece. Sin esa etiqueta las trayectorias son simplemente líneas sin bando y cualquier lectura táctica posterior (quién presiona, quién se repliega) se vuelve imposible. Hacerlo a mano no es una opción en un flujo automático que procesa miles de cuadros, así que hay que apoyarse en algún rasgo que distinga a

los dos equipos por sí solo. El más evidente es el color de la camiseta.

El problema es que el color tal como lo guarda una cámara no es tan estable como parece. La representación más habitual es el RGB, que describe cada píxel por la cantidad de rojo, verde y azul que lo compone. Sus tres números mezclan dos cosas que conviene separar: el color en sí y la cantidad de luz que recibe. Una camiseta roja a pleno sol y esa misma camiseta en la sombra de la grada tienen valores de RGB muy distintos, porque al bajar la iluminación caen los tres canales a la vez. El sistema ve dos rojos que numéricamente se parecen poco cuando para nosotros son obviamente el mismo.

Para esquivar esa fragilidad se recurre a espacios de color que apartan la luminosidad del cromatismo. El HSV organiza el color por tono, saturación y valor, de manera que el tono (lo que llamaríamos el color propiamente dicho) queda en una componente aparte y aguanta razonablemente bien los cambios de brillo. Dos tonos de la misma camiseta bajo luces distintas se mantienen cercanos en esa componente aunque su valor de luz difiera.

El CIELAB lleva esa idea más lejos y se diseñó pensando en cómo percibe el ojo humano las diferencias de color. Reserva un eje para la luminosidad y dos para el cromatismo, y tiene una propiedad valiosa que se conoce como uniformidad perceptual. Esto significa que la distancia geométrica entre dos colores dentro del espacio se corresponde de forma aproximada con el cómo de distintos los ve una persona. Esa correspondencia es la que permite definir la Delta E, una métrica que pone número a lo separados que están dos colores de un modo que tiene sentido para la vista y no solo para la aritmética [8]. Trabajar en CIELAB y medir con Delta E da una base mucho más firme para decidir si dos camisetas son del mismo color que comparar sus valores de RGB en crudo.

2.4.2 ❯ Agrupar sin etiquetas

Tener un buen espacio de color resuelve la mitad del problema. La otra mitad es separar a los jugadores en dos grupos sin disponer de ninguna etiqueta que diga de antemano quién es de cada equipo. Es un caso de agrupamiento no supervisado donde el algoritmo tiene que descubrir la estructura de los datos por su cuenta.

El método de referencia para esta tarea es K-means [25]. Consiste en fijar cuántos grupos se buscan y en colocar un centro provisional para cada uno. A partir de ahí repite dos pasos. Primero asigna cada punto al centro que tenga más cerca. Después, recalcula cada centro como el promedio de los puntos que le han tocado. Ese vaivén se mantiene hasta que las asignaciones dejan de moverse, momento en el que se considera que ha convergido. El resultado son grupos compactos en torno a sus centros que, en el caso del color, son las tonalidades predominantes de la escena.

La convergencia está garantizada pero no hacia la mejor solución posible. K-

means se detiene en un mínimo local que depende mucho de dónde se hayan puesto los centros al empezar. Una inicialización desafortunada puede dejar dos centros pegados dentro del mismo equipo y ninguno en el otro, con un reparto final pobre. Para mitigarlo se usa K-means++, una forma de sembrar los centros iniciales que los separa a propósito, eligiéndolos de manera que tiendan a caer lejos unos de otros [26]. Con esa siembra el algoritmo acaba con más frecuencia en una solución buena y suele necesitar menos iteraciones para llegar.

Llevado al campo el procedimiento se aplica sobre la región de la camiseta de cada jugador. Se recorta esa zona dentro de su caja, se agrupan los píxeles por color y el grupo más poblado da el color dominante de la prenda, ignorando de paso el césped del fondo o el dorsal impreso. Reuniendo el color dominante de todos los jugadores aparecen las dos agrupaciones grandes, una por equipo. Los porteros y el árbitro, que visten aparte, quedan como tonalidades que el sistema aparta como casos al margen.

2.4.3 › Cuándo se rompe

Todo lo anterior funciona mientras el color que llega a la cámara sea fiel al de la camiseta, y hay situaciones en que deja de serlo. El movimiento rápido produce desenfoque y una camiseta emborronada arrastra su color hacia el de lo que tiene alrededor, casi siempre el verde del campo. Algo parecido ocurre con los jugadores pequeños o lejanos, porque en el borde de su silueta cada píxel recoge a la vez parte de la camiseta y parte del césped, una mezcla óptica que tiñe de verde el color que el sistema cree estar midiendo. El color dominante se contamina y el reparto por equipos empieza a fallar.

La frontera de esta tarea pasa por dejar de mirar solo el color. En lugar del tono en crudo se emplean embeddings de apariencia, descriptores que una red neuronal aprende a partir de las imágenes y que resumen el aspecto de un jugador recogiendo bastante más que su color predominante. Son de la misma familia que los descriptores de reidentificación que ya aparecieron en el seguimiento [20], y al codificar texturas, formas y detalles del uniforme aguantan mejor el desenfoque y las mezclas con el fondo que hunden a los métodos basados únicamente en el color [3].

2.5 › Anclar la imagen al espacio: pose y puntos clave

2.5.1 › Localizar estructuras

Seguir a un jugador dentro de una caja y enlazar su recorrido a lo largo del tiempo cubre buena parte de lo que un análisis necesita pero no todo. Hay lecturas que dependen de algo más fino que la posición del cuerpo entero, de saber cómo está

colocado ese cuerpo en cada instante. Estimar la pose responde a esa necesidad. La tarea consiste en situar un conjunto fijo de puntos clave sobre cada persona detectada, casi siempre los que coinciden con sus articulaciones, de modo que el esqueleto resultante resuma la postura. En lo deportivo esa información alimenta el estudio de la técnica desde la mecánica de una zancada en carrera hasta el apoyo del pie en el instante del golpeo [3].

Las primeras soluciones con redes profundas trataron el problema como una regresión. La red recibía la imagen y devolvía directamente las coordenadas de cada articulación sin pasos intermedios. DeepPose fue el trabajo que abrió esa vía y encadenó varias etapas de refinamiento para corregir poco a poco una predicción inicial tosca [27]. Funcionaba, y demostró que una red podía con la tarea, pero predecir números seguidos se volvía impreciso en cuanto la postura se complicaba o aparecían varias personas mezcladas en la misma imagen.

El salto de calidad llegó al cambiar lo que la red tenía que producir. En lugar de las coordenadas se pasó a predecir un mapa de calor por cada articulación, que consiste en una imagen del mismo tamaño que la original donde el valor de cada píxel indica la probabilidad de que ahí esté ese punto. La posición final se lee en el máximo del mapa. La red en reloj de arena apilada popularizó este enfoque encadenando módulos que reducen la resolución de la imagen y la vuelven a ampliar varias veces seguidas, lo que permite combinar el detalle fino con el contexto global y afinar la estimación a lo largo del recorrido [28].

2.5.2 › Enfoques

Cuando hay varias personas en escena el problema se bifurca en dos estrategias con lógicas opuestas. La primera, llamada de arriba a abajo, detecta primero a cada individuo y estima la pose por separado dentro de su caja. Da buena precisión porque cada persona se procesa aislada pero su coste sube con cada jugador añadido, ya que hay que repetir la estimación tantas veces como detecciones haya. La segunda, de abajo a arriba, invierte el orden: localiza de una vez todas las articulaciones presentes en la imagen y luego las reparte entre las personas a las que pertenecen. Aquí el coste apenas depende de cuánta gente haya, lo que la hace eficiente cuando el campo está lleno.

OpenPose es el representante más conocido de la familia de abajo a arriba. Su aportación fue resolver el agrupamiento mediante campos de afinidad de partes, que son unos mapas que codifican la orientación de los miembros del cuerpo y permiten decidir qué articulaciones se conectan entre sí para formar cada esqueleto, todo ello en tiempo real y con un coste que no se dispara aunque crezca el número de personas [29]. En el lado de arriba a abajo la precisión depende mucho de conservar el detalle espacial, y ahí entra HRNet. Frente a las arquitecturas que comprimen

la imagen hasta una resolución baja y luego la reconstruyen, HRNet mantiene una representación de alta resolución durante todo el procesamiento, lo que mejora la localización de los puntos y la ha vuelto la base habitual sobre la que se montan muchos otros métodos [30].

La irrupción de los transformadores que se describió en la sección 2.2.3 [11] también alcanzó a la pose. ViTPose llevó esa arquitectura al problema con una propuesta simple, empleando un transformador visual corriente como espina dorsal y un decodificador ligero encima y, aun así, alcanzó resultados de primer nivel, lo que mostró que no hacía falta un diseño pesado para competir [31].

Hay una línea más que conviene destacar por lo bien que encaja con un sistema de una sola cámara, y es la que estima la pose en una única pasada. YOLO-pose extiende el detector YOLO presentado en la sección 2.2.4 [16] para que, además de la caja de cada persona, devuelva sus puntos clave en la misma pasada. Al asociar cada esqueleto a su caja desde el principio se ahorra el agrupamiento posterior de los métodos de abajo a arriba y las múltiples pasadas de los de arriba a abajo, lo que lo hace especialmente ligero [32]. Esa combinación de detección y pose en un solo modelo es la que mejor cuadra con un sistema que aspira a correr sobre vídeo de retransmisión sin una infraestructura pesada detrás.

Toda esta maquinaria se pensó para el cuerpo humano pero su utilidad aquí va más allá. Localizar las articulaciones de un jugador y localizar las marcas de un campo son, en el fondo, el mismo problema: el de encontrar un conjunto fijo de puntos clave dentro de una imagen. Un campo de fútbol tiene su propio grupo de puntos reconocibles en las esquinas, en los cruces de las líneas, en los vértices del área o en el punto de penalti. Reentrenar un detector de puntos clave para que busque esas marcas en lugar de codos y rodillas es lo que conecta la estimación de pose con el problema de situar la imagen sobre el plano del campo.

2.5.3 › Datasets y frontera

Detectar las marcas del campo sobre el vídeo de retransmisión arrastra una dificultad que la pose humana rara vez tiene. A una persona se la suele ver entera mientras que del terreno solo aparece un fragmento en cada cuadro, porque la cámara persigue el balón y cierra el plano sobre la jugada. Muchas de las marcas que harían falta para anclar la imagen quedan fuera del encuadre y, a menudo, hay que deducir dónde estaría una esquina que no se ve a partir de las pocas referencias que sí caen dentro. Es un problema de localización con información incompleta bastante más exigente que estimar un esqueleto sobre un jugador que ocupa el centro de la imagen.

Para medir el avance en estas condiciones se necesitaba un banco de pruebas a la altura, y ese papel lo cumple SoccerNet en su extensión con anotaciones espaciales. Ese conjunto etiqueta sobre miles de imágenes de emisión real las líneas del campo,

los postes de la portería y las marcas del terreno, y a partir de esas anotaciones habilita la tarea de calibración y localización del campo que sirve de referencia común para comparar métodos [33]. Disponer de una certeza anotada sobre retransmisión auténtica, con sus cortes de cámara y sus planos cerrados, es lo que ha permitido que el problema deje de tratarse caso por caso y se evalúe de forma sistemática.

La frontera actual ataca de lleno la cuestión de las marcas invisibles apoyándose en la geometría conocida del campo. El método PnLCalib es un buen exponente de esta dirección. Emplea dos redes de tipo codificador-decodificador, una que detecta los puntos clave y otra que detecta las líneas, y aprovecha que las dimensiones del campo están normalizadas para derivar de manera analítica la posición de puntos que no aparecen en la imagen, cruzando entre sí las líneas y las curvas que sí se han detectado [34]. Con ese conjunto ampliado de correspondencias entre la imagen y un modelo del campo se resuelve después la calibración, recuperando no solo la transformación plana sino los parámetros completos de la cámara.

2.6 › Del plano imagen al plano del campo: geometría proyectiva

2.6.1 › Modelo pinhole y homografía

Los puntos del campo que entrega la etapa anterior vienen en coordenadas de la imagen medidos en píxeles desde una esquina del fotograma. Para que sirvan a un análisis táctico hay que llevarlos a donde de verdad significan algo: sobre el césped y en metros, y eso exige un modelo de cómo la cámara ha convertido la escena real en esa imagen. El modelo de cámara estenopeica, o pinhole, es la descripción de partida. Supone que todos los rayos de luz que forman la imagen pasan por un único punto (el centro óptico) antes de proyectarse sobre el plano del sensor. Un punto del mundo y su imagen quedan así unidos por la recta que pasa por ambos y por ese centro, de modo que la posición en la imagen se obtiene viendo dónde corta esa recta al plano del sensor. Es una idealización que ignora la lente real y sus distorsiones pero captura la geometría esencial de la proyección.

Ese modelo se concreta en una cadena de transformaciones que conviene separar en dos bloques. El primero recoge lo que depende de la propia cámara y se agrupa en la matriz de parámetros intrínsecos, que suele escribirse K. Dentro están la distancia focal expresada en píxeles, que fija cuánto amplía la cámara, y el punto principal, que marca dónde el eje óptico atraviesa el sensor. El segundo bloque describe dónde está la cámara y hacia dónde mira. Lo forman una rotación R y una traslación t que llevan las coordenadas del mundo al sistema de referencia de la cámara. La proyección completa de un punto del espacio sobre la imagen se expresa entonces

como el producto de K por la combinación de R y t aplicada a ese punto, lo que en conjunto constituye la matriz de proyección de la cámara, de tamaño 3×4.

Para que esa concatenación de operaciones se escriba como un solo producto de matrices se recurre a las coordenadas homogéneas, que añaden una componente extra a las coordenadas habituales. Su ventaja no es solo de notación. La proyección en perspectiva no es una operación lineal sobre las coordenadas de píxel porque los objetos lejanos se ven más pequeños y eso supone una división por la profundidad. Al trabajar en coordenadas homogéneas esa división se reserva hasta el final, cuando se recuperan las coordenadas de la imagen dividiendo por la última componente, y mientras tanto toda la proyección queda como una multiplicación de matrices. Las coordenadas homogéneas traen además una propiedad que resulta central más adelante: representan el mismo punto aunque se multipliquen por cualquier factor distinto de cero, de manera que solo importan las proporciones entre sus componentes y no su escala absoluta.

Reuniendo ambos bloques en la matriz de proyección $\mathbf{P} = \mathbf{K}\,[\mathbf{R} \mid \mathbf{t}]$, un punto del mundo en coordenadas homogéneas se lleva a su imagen mediante

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K}\,[\mathbf{R} \mid \mathbf{t}] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}, \qquad \mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \tag{2.1}$$

donde f_x y f_y son las distancias focales en píxeles, (c_x, c_y) el punto principal y λ el factor de escala que recoge la profundidad y desaparece al dividir por la última componente.

El problema se vuelve mucho más manejable por una circunstancia particular del fútbol. Las marcas que interesan, las líneas pintadas y los puntos del terreno, están todas sobre una misma superficie plana. Cuando los puntos del mundo pertenecen a un plano se puede elegir el sistema de coordenadas de forma que ese plano tenga profundidad nula, y al hacerlo una de las columnas de la matriz de proyección deja de intervenir. Lo que queda es una matriz de 3×3 que relaciona directamente las coordenadas de un punto sobre el plano del campo con las de su imagen sin necesidad de reconstruir ninguna profundidad. Esa matriz es la homografía, y su existencia es la razón de que baste con geometría plana para resolver el problema en lugar de una reconstrucción tridimensional completa.

Al anularse la tercera coordenada del punto, la columna correspondiente de la rotación no interviene en el producto y la proyección se reduce a una transformación entre el plano del campo y la imagen,

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{H} \begin{pmatrix} X \\ Y \\ 1 \end{pmatrix}, \qquad \mathbf{H} = \mathbf{K}\,[\,\mathbf{r}_1 \;\; \mathbf{r}_2 \;\; \mathbf{t}\,], \tag{2.2}$$

donde $\mathbf{r}_1$ y $\mathbf{r}_2$ son las dos primeras columnas de $\mathbf{R}$ y $\mathbf{H}$ es la matriz 3×3 de la homografía.

Una homografía es una transformación proyectiva, así que mantiene las rectas como rectas y conserva una cantidad llamada razón doble, pero no preserva ni las distancias ni los ángulos ni el paralelismo. Dos líneas paralelas del campo, como las dos bandas, dejan de serlo en la imagen y se cortan en un punto de fuga, y el círculo central se ve como una elipse. Esto explica por qué no se pueden medir distancias reales directamente sobre el fotograma y hace falta deshacer la transformación para recuperar el plano del campo sin distorsión.

El número de incógnitas de la matriz tiene una particularidad heredada de las coordenadas homogéneas. La homografía tiene nueve entradas, pero como multiplicarla entera por una constante no cambia la transformación que representa, solo ocho de ellas son independientes. Esos ocho grados de libertad son los que hay que determinar a partir de correspondencias entre puntos del campo y puntos de la imagen. Cada correspondencia aporta dos ecuaciones, una por coordenada, así que con cuatro puntos que no estén alineados se reúnen las ocho ecuaciones necesarias para fijar la homografía por completo [35]. En un escenario real se utilizan muchos más de cuatro, porque las posiciones detectadas arrastran ruido y un excedente de correspondencias permite promediar ese error en lugar de confiar la solución a cuatro medidas imperfectas [8].

2.6.2 ❯ Resolver con robustez

El procedimiento clásico para calcular la homografía a partir de las correspondencias se conoce como transformación lineal directa o DLT. Su punto de partida es que la imagen de cada punto del campo y la predicha por la homografía deben coincidir salvo un factor de escala, que es la condición que se expresa de forma compacta mediante el producto vectorial entre ambas, ya que dos vectores proporcionales tienen producto vectorial nulo. Esa igualdad, aplicada a cada correspondencia, se reordena hasta dejar las entradas de la homografía como las incógnitas de un par de ecuaciones lineales. Apilando las ecuaciones de todas las correspondencias se forma un único sistema homogéneo en el que una matriz construida con las coordenadas de los puntos multiplica al vector que contiene las nueve entradas de la homografía e iguala a cero.

Escribiendo el punto del campo en homogéneas como $\mathbf{p}_i = (X_i, Y_i, 1)^\top$ y su imagen como (u_i, v_i), la condición de producto vectorial nulo aporta por cada co-

rrespondencia dos filas,

$$\begin{pmatrix} \mathbf{0}^\top & -\mathbf{p}_i^\top & v_i\,\mathbf{p}_i^\top \\ \mathbf{p}_i^\top & \mathbf{0}^\top & -u_i\,\mathbf{p}_i^\top \end{pmatrix} \mathbf{h} = \mathbf{0}, \tag{2.3}$$

con $\mathbf{h} = (h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}, h_{33})^\top$ el vector que recoge las nueve entradas de la homografía. Apilando las filas de los n puntos se obtiene el sistema homogéneo $\mathbf{A}\,\mathbf{h} = \mathbf{0}$.

Un sistema de ese tipo no se resuelve igualando a cero sin más porque la solución trivial de todo ceros no sirve y porque con datos ruidosos no existe una solución exacta. Lo que se busca es el vector que más se acerca a satisfacerlo sin ser nulo y ese vector lo proporciona la descomposición en valores singulares de la matriz del sistema. Entre todas las direcciones posibles la que minimiza el residuo es el vector singular asociado al menor valor singular, y esa es la homografía estimada. Hay un paso previo que decide buena parte de la calidad del resultado y que es fácil omitir, la normalización de los puntos antes de montar el sistema. Las coordenadas en píxeles toman valores de varios órdenes de magnitud, lo que produce una matriz muy mal condicionada en la que unas columnas pesan muchísimo más que otras y la solución se vuelve frágil ante el ruido. La normalización que propusieron Hartley y Zisserman traslada los puntos para que su centro quede en el origen y los escala para que su distancia media a ese origen sea constante. Se calcula la homografía con los puntos así transformados y luego se deshace la transformación. Equilibrar de ese modo la magnitud de las entradas mejora de forma notable la estabilidad del cálculo [35].

Resolver con todas las correspondencias a la vez es correcto mientras todas sean buenas, y ese es justo el supuesto que el fútbol no respeta. Los puntos los proporciona un detector que se equivoca, de manera que entre las correspondencias se cuelan emparejamientos falsos, una intersección de líneas confundida con otra parecida o una marca mal localizada en una zona embarrada. La estimación por mínimos cuadrados reparte el error entre todos los datos para minimizar el residuo global, y por esa misma razón una sola correspondencia gravemente equivocada arrastra la solución entera y deforma la homografía resultante.

RANSAC ataca ese problema invirtiendo la lógica del ajuste. En lugar de estimar con el conjunto completo, toma al azar una muestra mínima de cuatro correspondencias, calcula con ellas una homografía candidata y comprueba cuántas de las correspondencias restantes son coherentes con esa candidata contando como válidas o inliers, las que quedan dentro de un margen de error de reproyección fijado de antemano. El sorteo se repite muchas veces con muestras distintas sobre la idea de que basta con que una de ellas caiga libre de outliers para producir una hipótesis respaldada por el grueso de los puntos correctos. El número de repeticiones no se deja al azar sino que se calcula a partir de tres cantidades, la proporción esperada

de outliers, el tamaño de la muestra y la probabilidad con la que se quiere que al menos una muestra salga limpia, y crece deprisa a medida que aumenta la fracción de datos contaminados. Esas tres cantidades se relacionan en

$$N = \frac{\log(1-p)}{\log(1-(1-\epsilon)^s)}, \tag{2.4}$$

donde ϵ es la proporción de outliers, s el tamaño de la muestra mínima (cuatro para la homografía) y p la probabilidad buscada. De todas las candidatas se conserva la de mayor consenso y con el conjunto de inliers ya depurado se recalcula la homografía final empleando todas las correspondencias buenas a la vez, lo que combina la robustez del muestreo con la precisión de usar toda la información fiable [36].

2.6.3 › Estabilidad y frontera

Calcular la homografía de cada fotograma por separado introduce un defecto que solo se aprecia al ver el vídeo completo. Las posiciones de los puntos detectados se desplazan ligeramente de un cuadro al siguiente por el propio ruido del detector. La homografía estimada cambia un poco cada vez y esas pequeñas variaciones se trasladan amplificadas a la proyección, sobre todo en las zonas del campo alejadas de los puntos que han servido para calcularla donde la transformación está en cierto modo extrapolando. El efecto es que un jugador detenido aparece vibrando sobre el plano táctico no porque se mueva, sino porque la transformación que lo sitúa oscila entre fotogramas. A esa inestabilidad se la llama jittering proyectivo.

La forma de corregirlo se apoya en una observación simple sobre la cámara de retransmisión, y es que su movimiento es continuo y no da saltos. La secuencia de homografías a lo largo del tiempo debería ser igual de suave, así que filtrarla para que cada transformación sea coherente con las de los fotogramas vecinos elimina gran parte del temblor sin frenar el seguimiento del movimiento real de la cámara. Ese suavizado rinde mejor cuando se aplica sobre parámetros con sentido geométrico como la posición y la orientación de la cámara que cuando se promedian directamente las nueve entradas de la homografía, que están acopladas entre sí y no varían de manera independiente.

La homografía resuelve el problema mientras solo se trate de relacionar dos planos, y se queda corta en cuanto se pide algo más. No informa de dónde está la cámara en el espacio ni de cómo está orientada y pierde fiabilidad cuando en la imagen quedan pocas marcas sobre las que apoyarse, como ocurre en los planos muy cerrados sobre un jugador. La línea de trabajo más reciente recupera el modelo de cámara completo en vez de detenerse en la matriz de la homografía. PnLCalib, el mismo trabajo que generaba los puntos del campo en la sección 2.5.3 [34], plantea la calibración como una optimización no lineal sobre un modelo tridimensional del

terreno y devuelve los parámetros completos de la cámara, esto es, los intrínsecos K junto con la rotación R y la traslación t, en lugar de quedarse en la transformación plana. Para ese ajuste no se limita a los puntos clave, sino que aprovecha también las líneas detectadas e incorpora un módulo de refinamiento que parte de una calibración inicial y la mejora apoyándose en ellas, lo que le da robustez precisamente en las situaciones donde los puntos por sí solos no alcanzan.

2.7 › Integración y evaluación

2.7.1 › La tarea como problema formal

Las secciones anteriores han ido resolviendo piezas separadas: detectar a los jugadores, seguirlos, asignarles equipo y llevar sus posiciones al plano del campo. Juntar todas esas piezas en una sola tarea con una salida común es lo que se conoce como reconstrucción del estado del juego. A partir del vídeo de una única cámara, el objetivo es reconstruir un minimapa cenital en el que cada jugador aparece situado sobre el terreno con su rol y su equipo sin que nadie lleve ningún sensor encima. Es la tarea que engloba el sistema completo y la que de verdad refleja el problema que se quiere resolver.

Durante mucho tiempo no existió una forma estandarizada de medir un sistema así porque cada trabajo evaluaba sus módulos por su cuenta y comparar dos tuberías completas era casi imposible. SoccerNet-GSR vino a llenar ese vacío como primer banco de pruebas abierto de la tarea. Reúne doscientas secuencias de treinta segundos de vídeo de retransmisión anotadas con los puntos de las líneas para la calibración y con la posición sobre el campo de cada atleta junto a su rol, su equipo y su dorsal [37]. Disponer de esa referencia común permite por fin comparar sistemas de extremo a extremo en igualdad de condiciones.

Medir un sistema de este tipo exige una métrica que no se deje engañar. Las métricas de seguimiento más antiguas arrastraban un defecto conocido, y es que sobrevaloraban la detección o la asociación según cuál se mirase, de modo que un sistema podía parecer bueno acertando una de las dos y fallando la otra. HOTA se diseñó para corregir eso, equilibrando en una sola cifra la capacidad de encontrar a los objetos y la de mantener su identidad a lo largo del tiempo [38]. La métrica se descompone en dos factores: una precisión de detección que mide cómo de bien se localizan los jugadores y una precisión de asociación que mide cómo de bien se conservan sus identidades, y los combina mediante su media geométrica,

$$\mathrm{HOTA} = \sqrt{\mathrm{DetA} \cdot \mathrm{AssA}}. \tag{2.5}$$

La raíz del producto es lo que da a la métrica su carácter exigente, porque

obliga a acertar en las dos cosas a la vez. Un sistema que detecte de maravilla pero pierda constantemente las identidades hunde su factor de asociación y, como entra multiplicando, arrastra la puntuación entera por mucho que la detección sea impecable.

HOTA se pensó para el seguimiento sobre la imagen y mide la coincidencia entre cajas con el solape IoU, que no sirve cuando lo que importa no es la caja en píxeles sino la posición real sobre el césped. GS-HOTA adapta la métrica a la reconstrucción del estado del juego sustituyendo ese solape por una medida de similitud pensada para la tarea, en la que cuenta tanto lo cerca que está la posición estimada de la verdadera como el acierto en los atributos de identidad [37]. Esa similitud se expresa como el producto de un término de localización y otro de identidad,

$$\mathrm{Sim}_{\text{GS-HOTA}}(P,G) = \mathrm{LocSim}(P,G)\cdot\mathrm{IdSim}(P,G), \qquad \mathrm{LocSim}(P,G) = e^{\ln(0{,}05)\,\frac{\|P-G\|^2}{\tau^2}}, \tag{2.6}$$

donde $\|P-G\|$ es la distancia sobre el campo entre la posición predicha y la real, y τ fija la tolerancia, esto es, la distancia a la que la similitud de localización cae hasta 0,05. El término de identidad recompensa que coincidan el dorsal, el rol y el equipo. Al multiplicarse ambos, una posición correcta con la identidad equivocada, o al revés, queda penalizada igual que un fallo de seguimiento, que es justo lo que interesa premiar en un minimapa donde sirve de poco situar bien a alguien a quien se asigna el equipo contrario.

2.7.2 › Ensamblaje y modelos unificados

Construir un sistema de reconstrucción del estado del juego se aborda casi siempre montando una tubería modular en la que la salida de cada etapa alimenta a la siguiente. Un detector localiza a los jugadores en cada fotograma, un seguidor enlaza esas detecciones en trayectorias con identidad estable, un clasificador les asigna equipo y rol a partir del color o de descriptores de apariencia, la calibración estima la transformación que relaciona la imagen con el campo y una última proyección lleva cada posición al minimapa. Cada uno de esos eslabones es uno de los bloques desarrollados en las secciones anteriores, ahora encadenados.

Ese encadenamiento tiene una debilidad esencial, y es que los errores se propagan y se acumulan a lo largo de la cadena. Una detección que se pierde abre un hueco en la trayectoria que el seguidor tiene que rellenar a ciegas, y una calibración inestable desplaza todas las posiciones proyectadas por muy buena que haya sido la detección que las originó. El rendimiento final no lo marca el módulo más fino, sino el más débil, y por eso los desgloses de la métrica en su parte de detección y su parte de asociación resultan tan útiles para localizar dónde se está perdiendo la calidad.

El ecosistema abierto ha reducido bastante el coste de partida. Junto al dataset y la métrica, SoccerNet-GSR libera una implementación de referencia de extremo a extremo que ensambla todos esos componentes y funciona como punto de partida reproducible sobre el que construir [37]. Con ella y con las implementaciones abiertas de los detectores [16] y de los seguidores [21], [22], montar un sistema completo ya no obliga a programar cada pieza desde cero, sino a integrar y afinar componentes que existen y están probados.

Frente a la tubería por etapas avanza una alternativa que persigue resolver la tarea con un único modelo entrenado de principio a fin. Es la idea de los seguidores con transformador de la sección 2.3.3 llevada hasta sus últimas consecuencias, una red que aprende a producir el estado del juego directamente en lugar de delegar cada subproblema en un módulo distinto [24]. Su atractivo está en que, al optimizar todo conjuntamente, no hay errores que se transfieran de una etapa a otra. Su coste está en la cantidad de datos etiquetados y de cómputo que necesita y en una madurez que todavía no le permite superar a la aproximación modular en esta tarea concreta, donde la calibración por puntos y líneas sigue funcionando como una etapa propia incluso en los mejores sistemas [34].

2.7.3 › Referencias comerciales y el hueco

Al margen de la investigación académica existe un mercado de herramientas que persiguen lo mismo desde un enfoque comercial. Metrica Sports es un ejemplo de solución relativamente accesible, con un análisis a partir de vídeo, incluido el de una sola cámara, y un seguimiento semiautomático que resulta mucho más abordable que la instalación multicámara de los sistemas descritos en la sección 2.1 [39]. A cambio sigue siendo software cerrado, apoyado en revisión manual y sujeto a una licencia, de modo que ni es gratuito ni reproducible ni completamente automático.

El panorama, visto en conjunto, deja un hueco bastante definido. En un extremo están los sistemas profesionales multicámara, que entregan un dato muy preciso de los veintidós jugadores a cambio de una infraestructura fija y un presupuesto al alcance de muy pocos clubes [3]. En una posición intermedia se sitúan las herramientas comerciales semiautomáticas, que bajan la barrera de entrada pero no abren su código ni eliminan la intervención humana. Y en el lado de la investigación abierta está SoccerNet-GSR, que ha formalizado la tarea y ofrece un baseline reproducible, aunque su rendimiento todavía queda lejos del que exigiría un entorno de producción [37]. Entre esas tres realidades cabe un sistema de una sola cámara, automático y construido sobre componentes abiertos, que acerque el análisis posicional a quien no dispone de la infraestructura de un club grande.

CAPÍTULO 3

Objetivos

El objetivo general de este trabajo es construir un sistema capaz de extraer información táctica de un partido de fútbol partiendo de un único vídeo de retransmisión y funcionando sobre hardware de consumo. Dicho de otro modo, recuperar de la señal que ve cualquier espectador el dato posicional que la industria obtiene con instalaciones dedicadas y hacerlo de principio a fin desde el fichero de vídeo hasta una representación del juego sobre un plano del campo.

Ese propósito amplio se concreta en una serie de objetivos específicos que han ido guiando el desarrollo.

- Entrenar un detector capaz de localizar a los jugadores, al árbitro y al balón sobre fotogramas de retransmisión, con atención a los objetos pequeños y a las aglomeraciones de jugadores.
- Mantener la identidad de cada jugador a lo largo del tiempo mediante seguimiento multiobjeto de manera que su trayectoria pueda reconstruirse pese a las oclusiones y al movimiento de la cámara.
- Distinguir de forma automática a qué equipo pertenece cada jugador a partir del color de la camiseta, sin etiquetas previas, y separar a porteros y árbitro del resto.
- Calibrar el terreno de juego para relacionar la imagen con las coordenadas reales del campo y proyectar sobre un plano en dos dimensiones la posición de cada futbolista.

- Calcular a partir de esas posiciones un conjunto de métricas tácticas con sentido para el análisis del juego, como el centroide, la amplitud del bloque, los mapas de calor o la posesión.
- Integrar todas las piezas en una única tubería que procese un vídeo completo con la mínima intervención y deje su salida en un formato reutilizable.
- Evaluar el rendimiento del sistema con métricas objetivas sobre un conjunto de datos público y comprobar hasta qué punto los resultados se sostienen ante partidos y condiciones distintas.
- Publicar en abierto los modelos y el código para que el trabajo pueda reutilizarse y ampliarse.

Conviene dejar claro un límite desde el principio. El trabajo no persigue igualar la precisión de los sistemas comerciales multicámara ni funcionar en tiempo real, sino medir cuánto análisis táctico se puede recuperar desde la fuente más accesible y con los recursos más modestos. Esa frontera, la de lo que resulta posible sin infraestructura, es la que el sistema explora y la que las conclusiones del trabajo vuelven a poner sobre la mesa.

CAPÍTULO 4

Metodología

4.1 › Metodología de desarrollo

4.1.1 › Justificación de la metodología

El desarrollo de este trabajo se ha guiado por CRISP-DM. Es un modelo de proceso para proyectos de minería de datos independiente del sector de aplicación y de la tecnología empleada [40]. Su ciclo de vida se organiza en seis fases que van de la comprensión del problema al despliegue de la solución.

La elección responde a la naturaleza del trabajo, más cercano a un sistema dirigido por los datos y los modelos que a un desarrollo de software convencional. Su peso está en recolectar datos, prepararlos, entrenar modelos, evaluarlos y ponerlos a funcionar sobre vídeo real, la misma secuencia que recorren las fases de CRISP-DM.

4.1.2 › Las seis fases aplicadas al proyecto

El mapeo completo entre ambas estructuras aparece en la figura 4.1, que sitúa las ocho fases de desarrollo del proyecto sobre el ciclo de seis fases del modelo de referencia [40].

Las dos primeras fases acotan el problema y los datos disponibles. La comprensión del problema parte de los objetivos del trabajo, el análisis táctico extremo a extremo a partir de vídeo broadcast monocular, y queda recogida en el capítulo de objetivos. La comprensión de los datos explora los vídeos de partido y los conjuntos de SoccerNet de calibración y seguimiento atendiendo a sus formatos de anotación

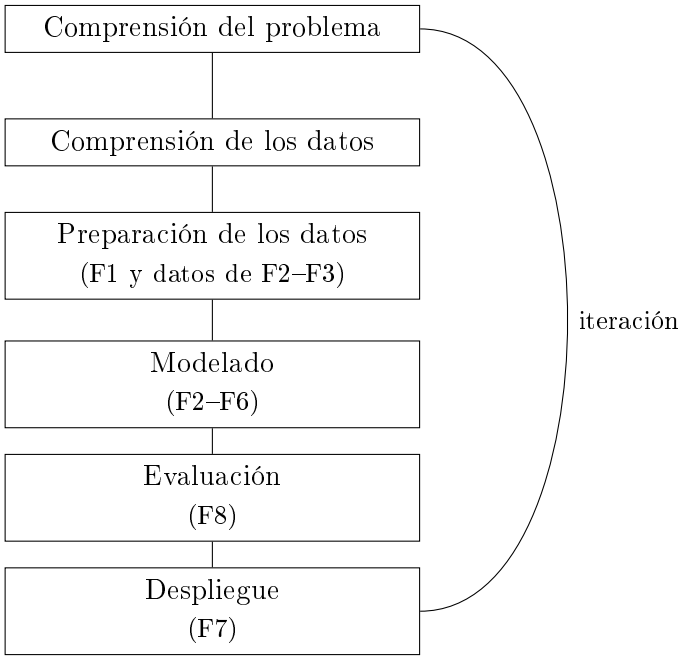


Figura 4.1: Fases de CRISP-DM y su correspondencia con las del proyecto. Fuente: elaboración propia a partir de Wirth y Hipp [40].

y a sus limitaciones. La preparación cubre desde el acondicionamiento del repositorio hasta el grueso del trabajo con datos, que incluye la extracción de frames, el auto-etiquetado, la revisión en Roboflow, el aumento de datos y la conversión de las anotaciones de SoccerNet al formato de keypoints de YOLO con revisión manual posterior en CVAT.

El modelado concentra el núcleo técnico y agrupa las líneas principales, la detección de objetos en sus versiones sucesivas, el modelo de keypoints del campo, la calibración con PnLCalib, el seguimiento con BoT-SORT, la clasificación de equipos por color en espacio CIELAB y el motor táctico. La evaluación mide el rendimiento del sistema con métricas cuantitativas de detección y seguimiento, una valoración cualitativa de la calibración y una prueba en lote sobre 49 clips. El despliegue cierra el ciclo con el pipeline integrado que procesa un vídeo completo modificando un único parámetro, su fichero JSON de salida y los modelos publicados en abierto.

4.1.3 ❯ Carácter iterativo y gestión de resultados negativos

CRISP-DM no impone una secuencia rígida. El orden de las fases depende del resultado de cada una y es habitual retroceder a tareas anteriores para repetirlas, hasta el punto de que sus autores recomiendan planificar las iteraciones de forma explícita [40].

En el proyecto ese carácter iterativo se concreta en cada paso y su motivo, y en la validación sobre un vídeo independiente al cerrar cada fase, que obliga a retroceder cuando los resultados no convencen. Los descartes razonados del desarrollo, el fine-

tuning del modelo de keypoints, ByteTrack frente a BoT-SORT, el OCR de dorsales, la corrección cinemática de intercambios de identidad y el auto-etiquetado no son callejones sin salida sino el resultado del bucle entre modelado y evaluación.

4.2 › Entorno de desarrollo

4.2.1 › Hardware

El sistema se ha desarrollado y ejecutado sobre una única máquina de gama de consumo. Cuenta con un procesador Intel Core i7-10750H de seis núcleos, 16 GB de memoria RAM, una tarjeta gráfica NVIDIA GeForce RTX 2060 con 6 GB de memoria de vídeo y sistema operativo Windows.

Esos 6 GB son una restricción real que ha condicionado decisiones a lo largo de todo el proyecto. Limitan el tamaño de modelo y el lote que caben en memoria y obligan a equilibrar la resolución de los vídeos. La detección trabaja a 1280 píxeles para ser precisa sobre objetos pequeños como el balón o los jugadores lejanos. Las alternativas más pesadas que no encajaban en ese presupuesto quedaron descartadas por este motivo.

4.2.2 › Lenguaje y librerías

El lenguaje es Python en su versión 3.13, elegido por su ecosistema de librerías de visión por computador y aprendizaje automático. El núcleo del sistema se apoya en PyTorch [41], el marco de aprendizaje profundo sobre el que se entrenan y ejecutan los modelos de detección y de keypoints.

Sobre esa base se articula el resto de la pila. La implementación de YOLOv8 usada para la detección y el seguidor BoT-SORT provienen de Ultralytics [42]. OpenCV se encarga de la lectura de imagen y vídeo, de las operaciones geométricas de la calibración y de la proyección homográfica. NumPy es el sustrato de cálculo numérico que hay debajo de todo. La asignación de equipos por color de camiseta emplea el algoritmo K-means de scikit-learn. SciPy y Matplotlib completan la pila para el filtrado y la visualización. La tabla 4.1 resume las versiones empleadas.

4.2.3 › Herramientas de soporte

El desarrollo se ha llevado a cabo en PyCharm con cuadernos de Jupyter, organizados por fases. El control de versiones y el código público residen en un repositorio Git alojado en GitHub.[1]

[1] https://github.com/HugoSanchezGallego/TFG

Componente	Versión
Python	3.13.13
PyTorch	2.6.0+cu124
CUDA	12.4
Ultralytics	8.4.53
OpenCV	4.13.0
NumPy	2.4.5
scikit-learn	1.9.0
SciPy	1.17.1
Matplotlib	3.10.9

Tabla 4.1: Pila de software y versiones empleadas.

El trabajo con datos y modelos se apoyó en tres servicios. Roboflow[2] gestionó el conjunto de detección y su revisión. CVAT[3] sirvió para revisar las anotaciones de keypoints. Los modelos entrenados se publican en abierto en el Hugging Face Hub[4] bajo licencia MIT.

4.3 › Conjuntos de datos

4.3.1 › Vídeos de la Bundesliga

Las imágenes para entrenar el detector proceden del DFL Bundesliga Data Shootout, una competición de Kaggle con grabaciones de partidos de la Bundesliga publicadas por la Deutsche Fußball Liga [43]. De esos vídeos se extrajeron fotogramas a intervalos regulares para construir el conjunto de detección de balón, jugador y árbitro. Las anotaciones se revisaron una a una en Roboflow hasta dejar un conjunto depurado de unas 1200 imágenes. De los mismos vídeos salieron también los fotogramas empleados en la parte de keypoints.

4.3.2 › SoccerNet-Calibration

El modelo base de detección de keypoints del campo se entrenó con SoccerNet-Calibration, el subconjunto de calibración del proyecto SoccerNet [33]. Contiene fotogramas de partidos profesionales de distintos estadios anotados con las líneas del terreno de juego. Como SoccerNet marca líneas y no puntos, esas anotaciones se transformaron a un conjunto fijo de 24 keypoints en formato YOLO-pose. El acceso a los datos de SoccerNet exige registrarse y aceptar un acuerdo de confidencialidad para obtener la contraseña de descarga.

[2] https://roboflow.com
[3] https://www.cvat.ai
[4] https://huggingface.co/hugosanchezgallego/futbol-detector-v5

4.3.3 › SoccerNet-Tracking

La evaluación se apoya en SoccerNet-Tracking, un conjunto de secuencias de fútbol anotadas con cajas e identificadores de tracklet en formato MOT que actúan como verdad de referencia [4]. Los clips de la partición de test, con su ground truth, son los que miden la detección, el seguimiento y la clasificación de equipos. La partición de entrenamiento aportó además fotogramas para ampliar el conjunto del detector manteniendo separadas las imágenes que entrenan de las que evalúan. El acceso sigue las mismas condiciones de SoccerNet del apartado anterior.

4.4 › Arquitectura del sistema

4.4.1 › Flujo del sistema

El sistema toma como entrada un único vídeo de retransmisión y devuelve una representación táctica en dos dimensiones. La figura 4.2 resume el flujo completo y la relación entre los módulos.

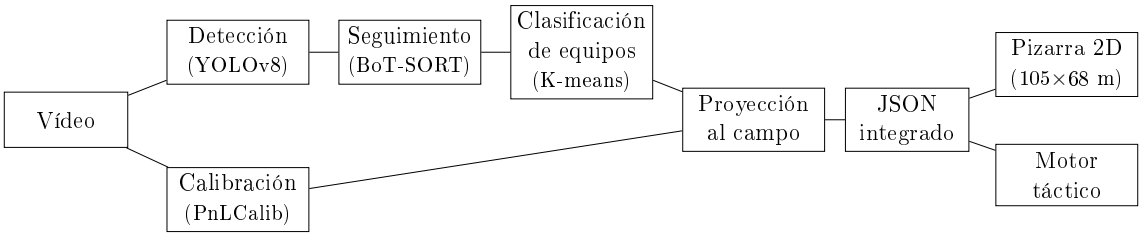


Figura 4.2: Arquitectura del sistema y flujo de datos entre módulos. Fuente: elaboración propia.

A partir del vídeo se abren dos caminos que trabajan en paralelo. El primero opera en el plano de la imagen y encadena la detección de objetos con YOLOv8, el seguimiento con BoT-SORT y la clasificación de cada jugador por equipo según el color de su camiseta. El segundo calibra el terreno de juego con PnLCalib y obtiene la homografía que relaciona la imagen con el modelo del campo.

Los dos caminos confluyen en la proyección, que sitúa cada objeto seguido y clasificado sobre las coordenadas reales del campo. Esa proyección alimenta tanto la pizarra táctica de 105 por 68 metros como el motor táctico, que calcula las métricas a partir de las posiciones.

4.4.2 › Interfaz de datos entre módulos

Los módulos no se intercambian imágenes sino una estructura de datos común que el pipeline va rellenando a medida que avanza y que el motor táctico consume al final. Esa estructura es un fichero JSON con el nombre del vídeo, el número de

fotogramas y una lista de fotogramas, donde cada fotograma es a su vez una lista con una entrada por objeto detectado.

Cada entrada reúne lo que han producido los distintos módulos sobre ese objeto. El identificador de seguimiento procede del tracker, el rol distingue entre balón, jugador, portero y árbitro, el equipo toma el valor 0 o 1 para los dos conjuntos y -1 cuando no aplica, la caja guarda la posición en píxeles de la imagen y el campo recoge las coordenadas en metros sobre la pizarra, vacías cuando la calibración de ese fotograma no es fiable. El siguiente fragmento muestra la forma de un fotograma con dos de esas entradas.

```
{
  "video": "soccernet_SNMOT-146.mp4",
  "n_frames": 750,
  "frames": [
    [
      {
        "id": 3,
        "rol": "jugador",
        "equipo": 0,
        "bbox": [812, 430, 845, 510],
        "campo": [54.2, 31.8]
      },
      {
        "id": 0,
        "rol": "balon",
        "equipo": -1,
        "bbox": [640, 455, 656, 471],
        "campo": [49.7, 33.2]
      }
    ]
  ]
}
```

4.5 › Metodología de evaluación

4.5.1 › Métricas de detección y seguimiento

La detección se mide con precisión, exhaustividad y su media armónica, la F1. La precisión penaliza las detecciones falsas, como un jugador inexistente o un balón donde no lo hay, y la exhaustividad penaliza las ausencias, como un jugador lejano

que pasa desapercibido. Una predicción se considera acierto cuando su caja solapa con la real por encima de un IoU de 0,5. El balón es el caso más exigente por su tamaño reducido y su movimiento rápido.

El seguimiento añade la dimensión temporal y se evalúa con dos métricas complementarias. La MOTA resume en un solo valor los fallos de detección, los falsos positivos y los cambios de identidad, por lo que queda muy condicionada por la calidad de la detección [44]. La IDF1 se construye sobre las trayectorias completas y mide con qué consistencia cada objeto conserva su identidad a lo largo del tiempo [45]. Esa consistencia es la que importa para el análisis táctico, donde seguir el recorrido de un jugador exige que mantenga el mismo identificador de principio a fin. El número de cambios de identidad se reporta además por separado porque cada uno parte en dos la trayectoria de un jugador.

La clasificación de equipos se mide con la exactitud, comparando el conjunto que asigna el sistema con el de la anotación sobre las detecciones que han encontrado su correspondencia con la verdad de referencia.

4.5.2 › Protocolo de evaluación

La verdad de referencia procede de SoccerNet-Tracking, que anota cada fotograma en formato MOT con el identificador y la caja de cada objeto y acompaña a cada secuencia de un fichero con el equipo y el rol de cada tracklet [4]. La salida del sistema se enfrenta contra esa anotación fotograma a fotograma. Todas estas métricas viven en el plano de la imagen, sobre las cajas anotadas, de modo que miden la pila de percepción con independencia de la proyección al campo.

La evaluación se organiza en dos niveles que persiguen objetivos distintos. El primero toma un único clip de 750 fotogramas y analiza en detalle todas las métricas sobre una misma secuencia, incluida la calibración, que solo se ejecuta aquí por su coste de cómputo. El segundo es una prueba en lote sobre 49 clips del conjunto de test que comprueba si los resultados se sostienen ante partidos, equipaciones y condiciones de cámara distintas. Esta segunda prueba trabaja en el plano de la imagen y prescinde de la calibración para poder cubrir muchas más secuencias.

4.5.3 › Evaluación de la calibración

La calibración del campo se valora de forma cualitativa y no con una métrica numérica. PnLCalib es un método de terceros que sus propios autores ya validaron de forma cuantitativa [34], así que lo que aquí se evalúa no es un algoritmo de calibración nuevo sino su integración dentro del sistema.

Reproducir una comparación métrica contra SoccerNet-Calibration tampoco sería metodológicamente sólido [33]. Esa medición volvería a evaluar el método sobre datos del mismo tipo con los que ya fue contrastado sin aislar lo que aporta el tra-

bajo, que es encadenar la calibración con el resto del pipeline. Por eso la calidad de la calibración se juzga por su efecto en la proyección. Se comprueba sobre la pizarra si las posiciones caen dentro de los límites del campo, si los dos equipos quedan en lados coherentes y si la trayectoria del balón resulta realista.

CAPÍTULO 5

Desarrollo y resultados

5.1 › Detección de objetos

El primer eslabón del sistema tiene que localizar en cada fotograma a los jugadores, al árbitro y al balón. Entrené para ello un detector YOLOv8 sobre un conjunto de datos que construí yo mismo con la dificultad de fondo de que el balón apenas ocupa unos píxeles y se pierde con facilidad sobre el césped.

5.1.1 › Construcción iterativa del dataset

No partí de un dataset cerrado sino de un puñado de imágenes que fui ampliando a medida que el propio detector mejoraba. El material base son doce vídeos de retransmisión del conjunto DFL Bundesliga Data Shootout de los que extraje un fotograma cada medio segundo. La primera versión del detector la etiqueté entera a mano en Roboflow, 152 imágenes con tres clases: balón, jugador y árbitro. Con tan pocos ejemplos el modelo daba resultados pobres pero ya servía para arrancar.

A partir de ahí el crecimiento fue automático y manual a la vez. Usaba el último modelo entrenado para pre-anotar fotogramas nuevos y después corregía las cajas en Roboflow. La primera ronda dejó claro el problema del balón porque el modelo solo localizó 170 de los cerca de 700 que había y tuve que añadir el resto a mano. Por esa vía llegué a unas 743 imágenes revisadas.

Para la versión definitiva amplié el material con el split de entrenamiento de SoccerNet-Tracking, 57 clips distintos de los que reservé para evaluar, de modo que ninguna imagen de test acabara colándose en el entrenamiento. Extraje fotogramas

espaciados, uno de cada veinte para no acumular imágenes casi idénticas, y reuní 1995. El detector de la etapa anterior los pre-anotó a 1280 píxeles, con más de treinta mil cajas propuestas, y los repasé en Roboflow corrigiendo lo que fallaba. De ese repaso conservé 455 imágenes, que sumé a las 743 ya revisadas. El conjunto resultante, al que llamo v5, reúne 1198 imágenes: 836 de entrenamiento, 247 de validación y 115 de test.

Sobre ese conjunto preparé además una variante con data augmentation. No la de giros y recortes que YOLO ya aplica sola durante el entrenamiento, sino transformaciones que imitan los defectos del vídeo de televisión, como la compresión JPEG agresiva y el desenfoque de movimiento en distintas direcciones (figura 5.1). No las repartí por igual. Generé copias aumentadas solo de las imágenes que contenían árbitro o balón, las clases con menos ejemplos, y dejé intactas las particiones de validación y test. El entrenamiento de esa variante pasaba de 836 a 2428 imágenes.

```python
def _jpeg(img):                              # compresión broadcast
    q = random.randint(45, 90)
    ok, enc = cv2.imencode(".jpg", img, [int(cv2.IMWRITE_JPEG_QUALITY), q])
    return cv2.imdecode(enc, cv2.IMREAD_COLOR) if ok else img

def _motion_blur(img):                       # movimiento rápido (balón/jugadores)
    k = random.choice([3, 5, 7])
    kernel = np.zeros((k, k), np.float32)
    if random.random() < 0.5: kernel[k // 2, :] = 1.0      # horizontal
    else:                     kernel[:, k // 2] = 1.0      # vertical
    kernel /= k
    return cv2.filter2D(img, -1, kernel)

def _gauss_blur(img):
    k = random.choice([3, 5])
    return cv2.GaussianBlur(img, (k, k), 0)

def _brightness_contrast(img):               # alpha=contraste, beta=brillo
    return cv2.convertScaleAbs(img, alpha=1.0 + random.uniform(-0.2, 0.2),
                                    beta=random.uniform(-25, 25))

def _gamma(img):                             # iluminación de estadio
    g = random.uniform(0.8, 1.2)
    lut = np.array([((i / 255.0) ** (1.0 / g)) * 255 for i in range(256)], np.uint8)
    return cv2.LUT(img, lut)

def _clahe(img):
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    l = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(l)
    return cv2.cvtColor(cv2.merge((l, a, b)), cv2.COLOR_LAB2BGR)

PIPELINE = [(_jpeg, 0.5), (_brightness_contrast, 0.5), (_gamma, 0.3), (_clahe, 0.2)]

def augmentar(img):
    out = img.copy()
    if random.random() < 0.3:                # como mucho un blur, 30% de las veces
        out = _motion_blur(out) if random.random() < 0.6 else _gauss_blur(out)
    for fn, p in PIPELINE:
        if random.random() < p:
            out = fn(out)
    return out
```

Figura 5.1: Celda del notebook 11 con el pipeline de aumento fotométrico, donde se aprecian la compresión JPEG agresiva y el desenfoque de movimiento. Fuente: elaboración propia.

El reparto de clases quedó muy desigual y no había forma de evitarlo (figura 5.2). En un partido hay veintidós jugadores sobre el campo, tres árbitros y un único balón, con lo que por cada caja de balón el conjunto reunía más de veinte de jugador. Ese desequilibrio reaparece luego en los resultados de cada clase.

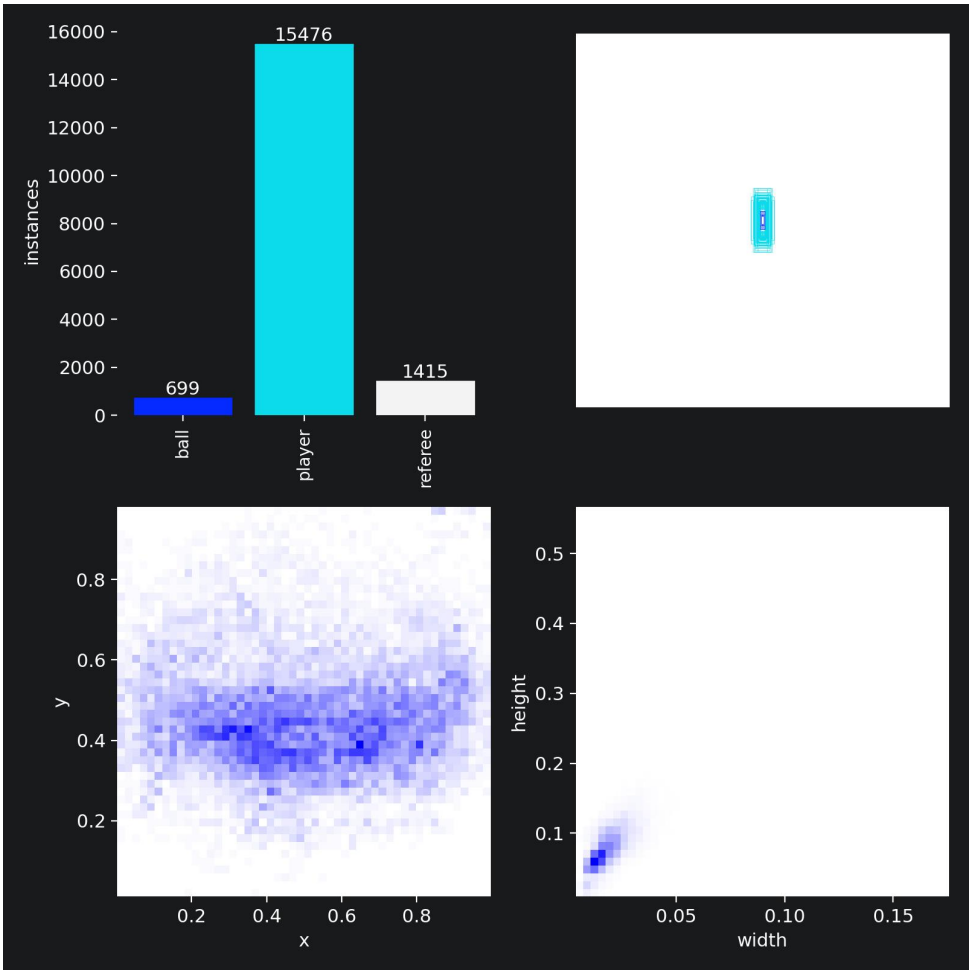


Figura 5.2: Reparto de instancias por clase y distribución del tamaño de las cajas del dataset. Deja ver de un vistazo el desbalance y lo diminuto que es el balón. Fuente: elaboración propia.

5.1.2 › Evolución de los modelos y entrenamiento

Entrené las sucesivas versiones cambiando una sola variable cada vez para poder atribuir cada mejora a su causa y no a una mezcla de cambios. La primera trabajaba con las 152 imágenes iniciales a 640 píxeles. La segunda mantenía esa resolución pero usaba ya las 743 revisadas. La tercera añadía data augmentation sobre ese mismo conjunto. La cuarta recuperaba el conjunto sin aumentar y subía la resolución de entrada a 1280.

Ese último cambio era el que más me interesaba por el balón. A 640 píxeles un objeto tan pequeño se reduce a una mancha que el modelo deja escapar mientras que a 1280 conserva algo de forma. Lo que se paga es cómputo porque cada imagen tarda más en procesarse y la velocidad de inferencia baja, y eso pesa cuando el objetivo final es recorrer un vídeo entero.

El detector definitivo, ya con el dataset v5 ampliado, arranca de los pesos pre-entrenados de YOLOv8s y se entrena cincuenta épocas a 1280 píxeles, con un lote reducido por la memoria de la tarjeta, una RTX 2060 de 6 GB (figura 5.3). Mantuve

la parada temprana en quince épocas sin mejora y dejé el optimizador en modo automático.

```python
from ultralytics import YOLO
model = YOLO("yolov8s.pt")   # mismo arranque que v4; o "best_v4.pt" para continuar desde él
results = model.train(
    data=r"C:\Users\sgghu\Downloads\dataset_v5\data.yaml",
    epochs=50, imgsz=1280, batch=2, device=0,
    patience=15, workers=0, cache=False, amp=True,
    project=r"C:\Users\sgghu\OneDrive\Documents\GitHub\TFG\runs\detect\models",
    name="detector_v5_1280",
)
[6]
```

Figura 5.3: Celda de configuración y lanzamiento del entrenamiento, con los hiperparámetros (`epochs`, `imgsz`, `batch`, `optimizer`). Fuente: elaboración propia.

En esas curvas (figura 5.4) las tres pérdidas bajan de forma sostenida y la precisión y el recall suben sin que la validación se despegue de la de entrenamiento, señal de que el modelo generaliza en lugar de memorizar el conjunto.

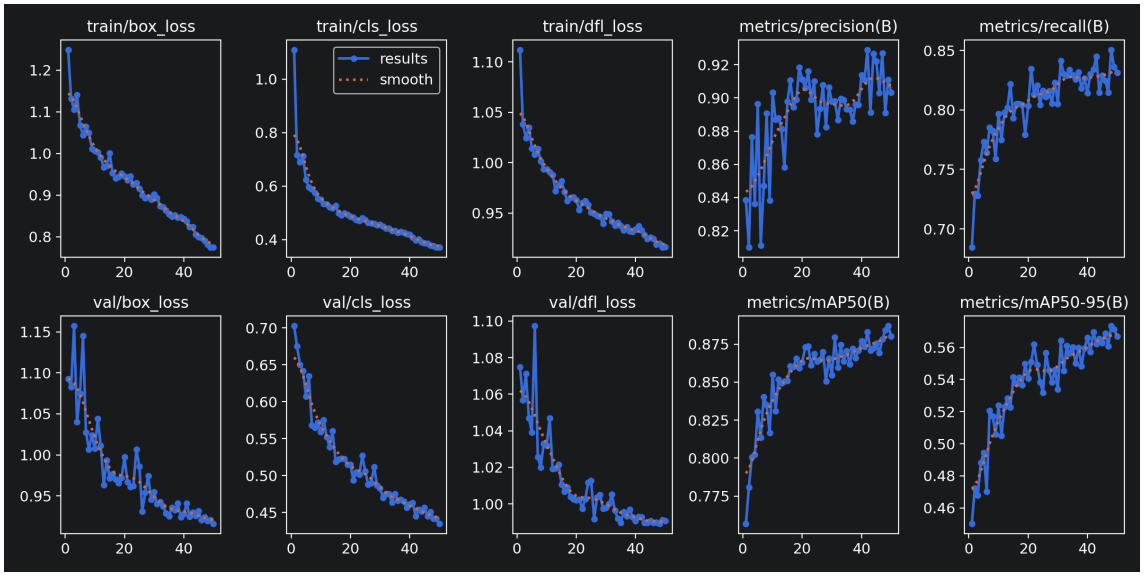


Figura 5.4: Curvas de entrenamiento y validación a lo largo de las cincuenta épocas. Fuente: elaboración propia.

5.1.3 ❱ Resultados y selección del modelo

Para comparar las cuatro primeras versiones en igualdad de condiciones las evalué todas sobre el mismo conjunto de test, 74 imágenes que ningún modelo había visto, con 1782 objetos anotados. La tabla 5.1 reúne sus métricas.

El mayor salto lo dan los datos. Pasar de 152 a 743 imágenes sube el mAP global de 0,48 a 0,64, y el árbitro es quien más lo nota, de ser casi indetectable a rebasar el 0,5. La data augmentation a 640 píxeles aporta poco al promedio, alrededor de un punto, aunque sigue empujando al árbitro. El balón solo se mueve con la resolución.

Modelo	Balón	Jugador	Árbitro	mAP50	mAP50-95	FPS aprox.
v1 (152 img, 640 px)	0,398	0,893	0,143	0,478	0,212	79
v2 (743 img, 640 px)	0,445	0,949	0,516	0,637	0,350	84
v3 (743 + aug, 640 px)	0,466	0,929	0,566	0,654	0,350	57
v4 (743 img, 1280 px)	0,574	0,962	0,581	0,706	0,385	31

Tabla 5.1: Comparativa de los detectores v1–v4 sobre el mismo conjunto de test (74 imágenes, 1782 objetos). Fuente: elaboración propia.

A 1280 píxeles su mAP sube de 0,45 a 0,57 y tira del global hasta 0,71, al precio de que la velocidad caiga de 84 a 31 fotogramas por segundo.

El balón fue la clase difícil de principio a fin y nunca se acercó a las otras dos. Pesan tres cosas a la vez: los pocos ejemplos disponibles, su tamaño minúsculo y el desenfoque que arrastra al moverse rápido. La matriz de confusión del modelo final lo retrata bien (figura 5.5), porque sus fallos no son confusiones con jugadores sino balones que se pierden contra el fondo.

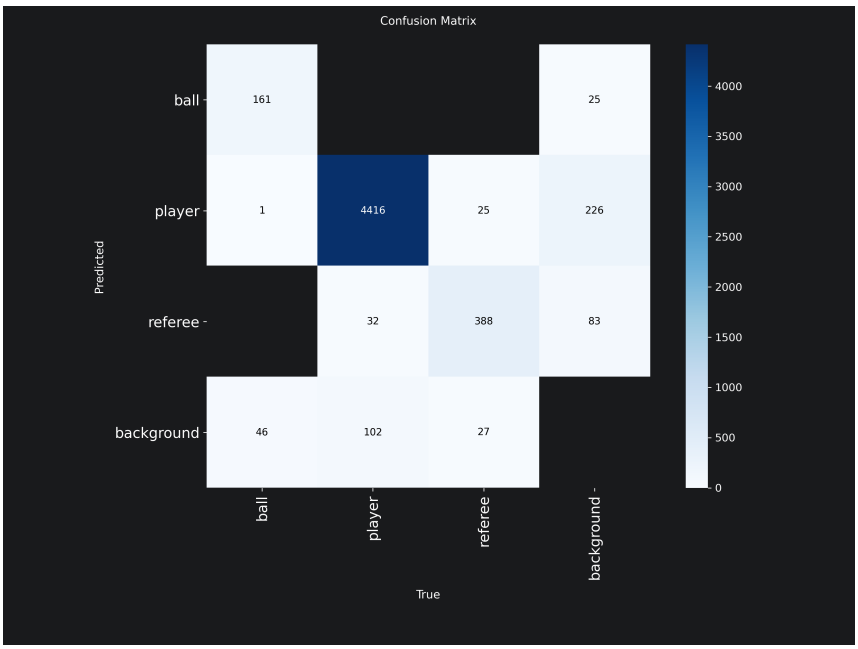


Figura 5.5: Matriz de confusión del detector final. Sus fallos en el balón no son confusiones con el jugador sino balones que se pierden contra el fondo. Fuente: elaboración propia.

Con el dataset v5 entrené el detector definitivo, evaluado esta vez sobre su propia partición de validación, 247 imágenes con bastantes más instancias que el test anterior. Sus cifras no son directamente comparables con las de la tabla anterior y por eso van por separado (tabla 5.2).

Sobre ese conjunto el detector alcanza un mAP de 0,884 (figura 5.6). La variante con augmentation dirigida deja el promedio casi igual, 0,890, con el cambio que buscaba al generarla: el árbitro sube de 0,894 a 0,922, mientras el balón y el jugador ceden una décima escasa. La mejora del árbitro no compensaba sumar al pipeline

Modelo	Balón	Jugador	Árbitro	mAP50	F1 (conf. óptima)
v5 (dataset ampliado)	0,775	0,984	0,894	0,884	0,86 (0,331)
v5 + data augmentation	0,768	0,979	0,922	0,890	0,88 (0,424)

Tabla 5.2: Resultados del detector v5 sobre su partición de validación (247 imágenes). Fuente: elaboración propia.

un modelo entrenado sobre el doble de imágenes, así que me quedé con el v5 sin esa augmentation añadida.

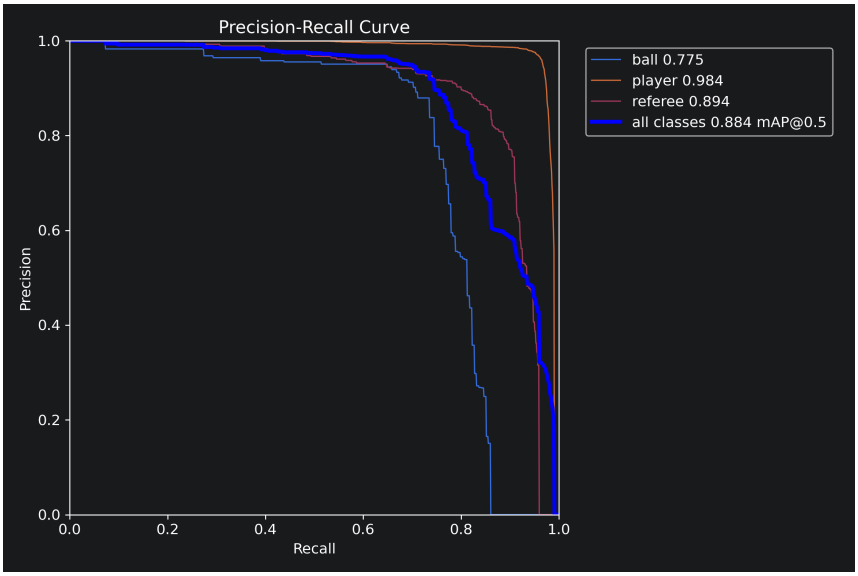


Figura 5.6: Curva precisión–recall del detector v5 con el mAP por clase. Fuente: elaboración propia.

El resto del sistema, que desarrollo en las secciones siguientes, lo construí sobre v4, el mejor detector de la comparativa inicial. La ampliación del dataset llegó más tarde, y como v5 rendía por encima de v4 con holgura, es el detector que el sistema emplea en la evaluación cuantitativa final y el que dejé publicado en abierto en Hugging Face, junto al v4 que ya tenía subido.

5.2 › Seguimiento y clasificación de equipos

El detector entrega en cada fotograma un conjunto de cajas sueltas sin memoria de un cuadro al siguiente. Para analizar el juego necesito que cada jugador conserve una identidad estable a lo largo del vídeo y que esa identidad lleve asociado su equipo y su papel sobre el campo. Esas dos cosas, seguir y clasificar, son las que resuelvo en esta fase partiendo del detector v4 que ya tenía montado.

5.2.1 › Seguimiento de jugadores

El seguimiento multiobjeto enlaza las detecciones de fotogramas consecutivos en trayectorias con identificador propio. En el fútbol esto se complica por las aglomeraciones porque cuando varios jugadores se juntan y se tapan entre sí el seguidor pierde el hilo y, al reaparecer, tiende a repartir identificadores nuevos.

Probé dos algoritmos. ByteTrack asocia las cajas entre fotogramas por solapamiento y puntuación de confianza, sin mirar la apariencia. Es rápido y se comporta bien mientras los jugadores están separados, pero en las aglomeraciones intercambia identidades con facilidad. BoT-SORT parte de la misma idea y le añade dos cosas que en este vídeo importan, una reidentificación por apariencia que compara el aspecto de cada jugador para recuperar su identificador tras una oclusión y una compensación del movimiento de la cámara que descuenta los paneos y zooms de la retransmisión antes de predecir dónde estará cada uno. Lo configuré con un buffer de track largo (ciento veinte fotogramas, unos cinco segundos) para que una identidad sobreviva a oclusiones prolongadas sin darse por perdida (figura 5.7).

```
botsort_cfg = REPO_ROOT / "configs" / "botsort_custom.yaml"
botsort_cfg.parent.mkdir(parents=True, exist_ok=True)

contenido = """tracker_type: botsort
track_high_thresh: 0.25
track_low_thresh: 0.1
new_track_thresh: 0.6
track_buffer: 120
match_thresh: 0.8
fuse_score: true
gmc_method: sparseOptFlow
proximity_thresh: 0.5
appearance_thresh: 0.25
with_reid: true
model: auto
"""

with open(botsort_cfg, "w", encoding="utf-8") as f:
    f.write(contenido)

print(f"Config creada en: {botsort_cfg}")
print(f"Existe: {botsort_cfg.exists()}")
[2]
```

Figura 5.7: Celda donde se genera la configuración de BoT-SORT, con `track_buffer: 120`, `gmc_method: sparseOptFlow` y `with_reid: true`. Fuente: elaboración propia.

Me quedé con BoT-SORT, que mantiene las identidades de forma sensiblemente más estable que ByteTrack: este último, sin reidentificación ni compensación de cámara, reparte más identificadores nuevos en cuanto se forma una aglomeración.

Sobre el clip de prueba la duración media de cada trayectoria de jugador rondaba los trescientos fotogramas (figura 5.8).

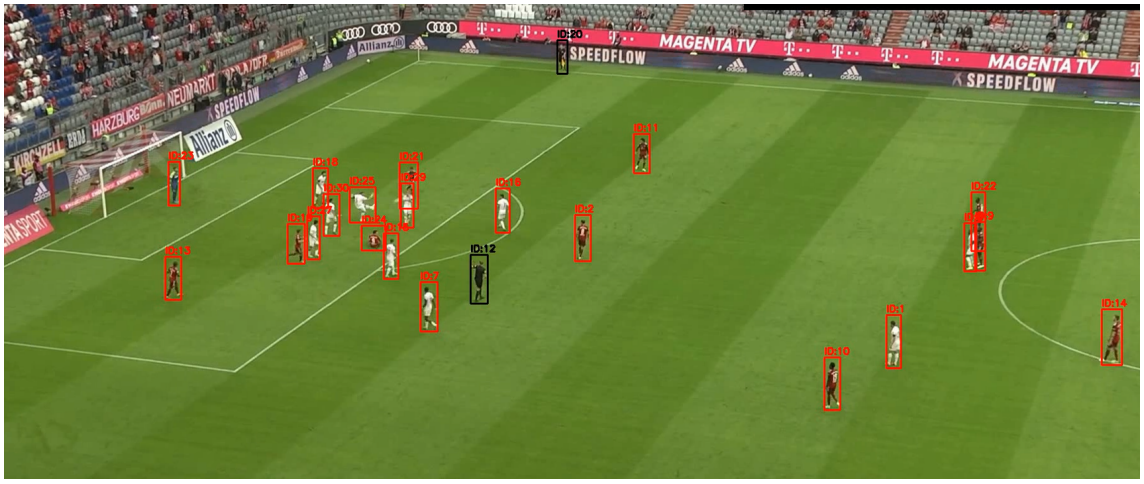


Figura 5.8: Fotograma del seguimiento con las cajas y el identificador de cada jugador, con una aglomeración a la vista. Fuente: elaboración propia.

El balón lo trato aparte y no entra en el seguidor. No me hace falta darle una identidad persistente porque solo hay uno, así que una instancia separada del detector lo localiza fotograma a fotograma y su trayectoria se depura después descartando saltos imposibles e interpolando los huecos cortos. Aun así es el elemento que peor se detecta, herencia directa de lo que ya vimos en la detección, y su cobertura exacta, en crudo y tras la interpolación, la doy en la evaluación cuantitativa.

5.2.2 › Post-procesado de los tracklets

Por bueno que sea el seguidor las oclusiones siguen fragmentando alguna trayectoria, y un jugador que desaparece tras un grupo y vuelve a aparecer recibe a veces un identificador nuevo. Como el análisis es offline y dispongo del vídeo entero, puedo permitirme un post-procesado que un sistema en tiempo real no haría.

Lo abordo en dos pasos. Primero divido por clase con histéresis: si la clase de un track cambia de forma sostenida durante varios fotogramas y no en un parpadeo aislado, lo parto en dos, de modo que un mismo identificador no acabe mezclando a un jugador con un árbitro. Después coso los tracklets. Cuando una trayectoria muere y otra nace pocos fotogramas más tarde en una posición compatible con la velocidad que llevaba la primera, fusiono ambos identificadores en uno solo. Los umbrales aquí son conservadores a propósito porque fundir por error a dos jugadores distintos es un fallo peor que dejar una trayectoria partida en dos trozos (figura 5.9).

El efecto es una limpieza notable del recuento de identidades. En el clip de validación las de jugador bajan de 43 a 26, y las de árbitro de siete a dos, que son las dos que de verdad aparecen en pantalla.

```
candidatos = []
for ta, ra in resumen.items():
    for tb, rb in resumen.items():
        if ta == tb or clase2[ta] != clase2[tb]:
            continue
        gap = rb["ini"] - ra["fin"]
        if gap > MAX_GAP or gap < -MAX_SOLAPE:
            continue
        if gap > 0:
            pos_predicha = ra["pos_fin"] + ra["vel_fin"] * gap
            dist = np.linalg.norm(rb["pos_ini"] - pos_predicha)
            radio = RADIO_BASE + RADIO_POR_FRAME * gap
        else:
            # Solapamiento: comparar posición de A en el frame en que nace B
            pa = pos_por_frame[ta].get(rb["ini"])
            if pa is None:
                continue
            dist = np.linalg.norm(rb["pos_ini"] - pa)
            radio = RADIO_SOLAPE
        if dist <= radio:
            candidatos.append((dist, ta, tb))
```

Figura 5.9: Núcleo del cosido de tracklets: un track que muere se empareja con otro que nace en una posición compatible con su velocidad. El post-procesado reduce el recuento de 43 a 26 jugadores y de siete a dos árbitros. Fuente: elaboración propia.

Tanteé un tercer paso, una corrección cinemática que detectara intercambios de identidad entre jugadores cercanos comparando sus trayectorias. Lo descarté porque el único intercambio que llegó a marcar como corregible resultó ser un falso positivo, así que añadía riesgo sin aportar nada. Lo dejé en el cuaderno como registro comentado.

5.2.3 › Clasificación de equipos y roles

El seguimiento reparte identidades pero no sabe a qué equipo pertenece cada una. La asignación la hago por el color de la camiseta. Para cada detección recorto la franja del torso, descarto entre el veintidós y el cincuenta por ciento de la altura de la caja para evitar la cabeza y las piernas, elimino los píxeles de césped con un filtro de verdes y extraigo el color dominante de lo que queda con un pequeño K-means, cuyo grupo mayoritario es la camiseta. A cada identidad le asigno la mediana de los colores de todas sus apariciones, lo que la hace inmune a un fotograma puntual en el que el recorte salga mal (figura 5.10).

La separación en dos equipos la hace un K-means con k igual a dos, pero no sobre el color tal cual, sino sobre los canales a y b del espacio Lab. Trabajar en Lab y quedarme solo con esos dos canales descarta la luminancia, de manera que las sombras y los cambios de iluminación dejan de confundir la agrupación, que pasa

a depender únicamente del matiz. Cada identidad se asigna al equipo que vota la mayoría ponderada de sus detecciones, dando más peso a las de caja grande, donde el color es más fiable. Las identidades que quedan lejos de los dos centroides las marco como porteros, que visten una equipación que no es la de ninguno de los dos conjuntos.



Figura 5.10: Color mediano de camiseta extraído para cada identidad seguida. Sobre estos colores, en el plano a–b de Lab, el K-means separa los dos equipos. Fuente: elaboración propia.

Llegar a esto me costó dos intentos antes. El primero promediaba el color del torso sin más y mezclaba camiseta con piel y sombras, de forma que un rojo salía amarronado y un blanco beige. El segundo, ya con la mediana, agrupaba sobre la cromaticidad en RGB y seguía fallando con los jugadores lejanos, cuyo color se funde con el verde del campo por el desenfoque. El cambio al plano a-b de Lab fue lo que estabilizó la clasificación.

Ese plano a-b es el espacio en el que ajusté y validé la clasificación en esta fase. Más adelante al integrar el pipeline completo y en la evaluación final, pasé a clasificar directamente en BGR, que separa mejor un rojo oscuro de un blanco porque conserva el brillo que el plano a-b descarta.

Queda el papel de cada uno, y ahí el color no ayuda. Distinguir a un portero de un juez de línea es imposible mirando solo los píxeles porque pueden vestir de un color parecido y nada en su apariencia los separa. Lo resuelvo por su posición sobre el campo una vez que la calibración de la sección siguiente me da las coordenadas de campo de cada identidad. Un portero se reconoce porque vive pegado a su área, y de hecho el sistema corrige la asignación de equipo de un portero cuando su cercanía a una de las porterías contradice el color que se le había estimado.

La validación la hice sobre un vídeo que no usé en ningún momento del desarrollo, con equipaciones distintas a las del clip con el que ajusté el método. Etiqueté a mano una muestra aleatoria de cien detecciones y comparé el equipo asignado con el real, con una precisión del 95 % (figura 5.11).



Figura 5.11: Muestra aleatoria de cien detecciones con el equipo asignado en el color del borde, sobre la que se mide a mano la precisión de la clasificación en un vídeo independiente. Fuente: elaboración propia.

5.3 › Calibración del campo y proyección a la pizarra

Hasta este punto todo ocurre en el plano de la imagen. Para hablar de posiciones, distancias o zonas necesito pasar cada detección a coordenadas reales del campo y para eso tengo que saber cómo se proyecta el terreno sobre la cámara. Esa transformación es la homografía. Casi toda la fase consiste en obtenerla con la mayor fiabilidad que se pueda sacar de un único plano de televisión.

5.3.1 › Modelo de keypoints base

Entrené un modelo de pose de YOLOv8 para encontrar veinticuatro puntos de referencia del campo, desde las esquinas y los vértices de las áreas hasta los extremos del círculo central. SoccerNet trae esas marcas anotadas como líneas y no como puntos, así que escribí una conversión que toma cada cruce relevante y lo guarda como un keypoint con su coordenada conocida sobre un campo de 105 por 68 metros. El entrenamiento se apoya en los 10.696 fotogramas de SoccerNet, partidos profesionales grabados en estadios muy distintos.

Para evaluarlo necesitaba imágenes que no hubiera visto nunca. Aparté cincuenta fotogramas de Bundesliga, los autoetiqueté con el propio modelo y los corregí en CVAT. Pocos, pero ninguno presente en el entrenamiento. La figura 5.12 recoge sus detecciones sobre varios de esos fotogramas.



Figura 5.12: Puntos de referencia del campo detectados por el modelo base sobre fotogramas de prueba. Fuente: elaboración propia.

5.3.2 › Un fine-tuning en paralelo

Mientras avanzaba con el resto de la calibración probé a especializar el modelo en el dominio concreto de la Bundesliga. Junté noventa y seis fotogramas etiquetados a mano y reentrené el base con varios grados de congelación de capas, desde reajustar la red entera hasta dejar fija casi toda y mover solo la cabeza.

Los resultados en validación animaban. Las versiones afinadas llegaban a un mAP50 de 0,95, bastante por encima de lo que rendía el base en ese mismo conjunto.

El test independiente contó otra historia. La variante que mejor puntuaba en validación, la del backbone congelado, se quedaba en 0,690 sobre los cincuenta fotogramas reservados mientras el base aguantaba en 0,806 (tabla 5.3). Con tan pocas

imágenes el ajuste se había pegado a la validación. Era una prueba lateral, así que me quedé con el base y no insistí.

Modelo	Congelación	mAP50 val.	mAP50 test
A (base SoccerNet)	sin FT	n. d.	0,806
B (completo)	freeze=0	0,940	0,801
C (backbone)	freeze=10	0,951	0,690
D (cabeza)	freeze=22	0,949	0,652
E (backbone, 1280)	freeze=10	0,718	0,741

Tabla 5.3: En validación las variantes afinadas parecen superar al base. Sobre el test independiente el base (0,806) las iguala o supera. Fuente: elaboración propia.

5.3.3 › De los keypoints a la homografía

Cuatro puntos emparejados bastan para resolver la homografía. El modelo suele dar más pero no todos sirven igual. El punto central y los dos de las medias del campo se apoyan en una raya larga y recta sobre la que la red acierta la altura pero falla mucho en dónde cae el cruce a lo ancho, así que aparté esos tres (los keypoints 14, 15 y 16) del cálculo (figura 5.13).

```
# Keypoints poco fiables que excluimos de la homografía (centro y medias)
EXCLUIR = {14, 15, 16}  # centro, media_inf, media_sup

# Emparejar solo keypoints visibles Y fiables
idx_validos = [i for i in range(24) if visibles[i] and i not in EXCLUIR]
src_pts = kps_xy[idx_validos]
dst_pts = KP_REALES[idx_validos]

print(f"Puntos para la homografía (sin centro/medias): {len(src_pts)}")

if len(src_pts) < 4:
    print("ERROR: se necesitan al menos 4 keypoints visibles")
else:
    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 3.0)
    inliers = mask.sum() if mask is not None else 0
    print(f"Homografía calculada")
    print(f"Inliers: {inliers} de {len(src_pts)}")
[5]
```

Figura 5.13: Fragmento que aparta los keypoints del centro y las medias (14, 15 y 16) antes de calcular la homografía. Fuente: elaboración propia.

El descarte cambia el resultado por completo. Con esos puntos dentro, la reconstrucción del campo arrastraba un error medio de reproyección de 10,15 metros, de sobra para mandar a los jugadores lejos de su sitio. Sin ellos baja a 0,65 metros, del orden del grosor de las propias líneas. La figura 5.14 pone las dos proyecciones una al lado de la otra.

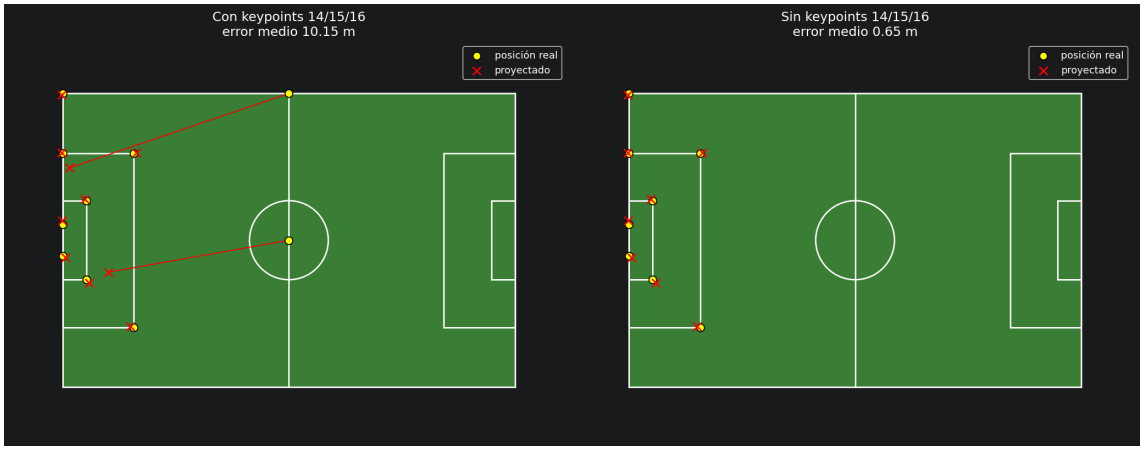


Figura 5.14: Campo proyectado manteniendo los keypoints 14, 15 y 16 (error de 10,15 m) frente a excluyéndolos (0,65 m). Fuente: elaboración propia.

5.3.4 ❯ La cobertura, el punto débil de mi modelo

El problema de mi modelo de keypoints no estaba en la precisión sino en la cobertura. La realización televisiva cierra el plano sobre el balón y deja fuera de cuadro buena parte del campo, de modo que en muchos fotogramas no llegan a verse cuatro keypoints. Repasando un clip completo, solo en torno a la mitad reunían keypoints suficientes para una homografía propia. La otra mitad se quedaba sin forma de proyectar.

El primer intento de tapar esos huecos fue propagar la última homografía buena con el movimiento de la cámara. Mantenerla congelada no sirve con un plano que no para de moverse, porque la pizarra se queda quieta mientras el encuadre viaja. Sigo con flujo óptico un conjunto de puntos del fondo de un fotograma al siguiente, calculo la transformación que los relaciona y la encadeno con la última homografía real, y un RANSAC descarta los puntos que se mueven distinto al fondo, que son los jugadores. Sobre el clip de validación esto llegaba a cubrir los 749 fotogramas, 373 con keypoints reales y 376 con la homografía propagada (figura 5.15).

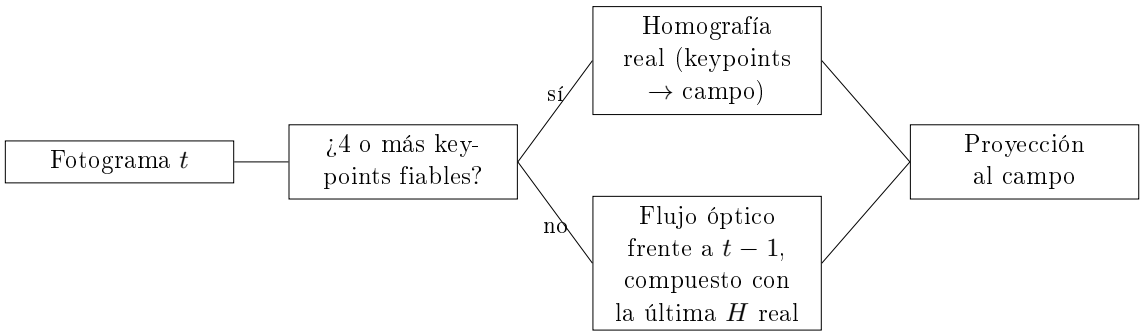


Figura 5.15: Primer enfoque, basado en mi modelo de keypoints. Para cada fotograma, si hay cuatro o más keypoints fiables se calcula la homografía real; si no, se propaga la última válida con el movimiento de cámara estimado por flujo óptico. Fuente: elaboración propia.

El inconveniente es que la propagación encadena estimaciones, y en cuanto la cadena se alarga el error de cada paso se acumula y la proyección se va de su sitio. Servía para tramos cortos, pero no daba una base lo bastante fiable para todo el vídeo.

5.3.5 › El cambio a PnLCalib

Tras ver esos fallos pasé a PnLCalib, un calibrador de terceros pensado para vídeo de difusión. No depende de un grupo de puntos aislados. Combina una red de keypoints con otra que detecta las líneas del campo y ajusta los parámetros de la cámara contra unos y otras, así que resuelve incluso cuando el cruce exacto no aparece pero sí un trozo de línea. Sobre el clip de validación calibra los 749 fotogramas sin apoyarse en la propagación.

A cambio hay que vigilarlo. Cada cierto tiempo suelta una calibración disparatada que manda las posiciones lejísimos del rectángulo de juego, y esas las descarto comparándolas con los límites del campo. PnLCalib es la calibración que monta el pipeline integrado y la única que uso en la evaluación, porque es la que considero correcta. Mi modelo de keypoints queda como el primer intento que me llevó hasta ella.

5.3.6 › Proyección a la pizarra

Con la homografía de cada fotograma proyecto cada objeto detectado. Uso el punto de los pies, que tomo en el centro del borde inferior de la caja, y lo traslado al campo. Esas coordenadas en metros son las que dibujo en la pizarra, una vista cenital del terreno donde cada jugador es un punto del color de su equipo y el árbitro y el balón van aparte (figura 5.16).

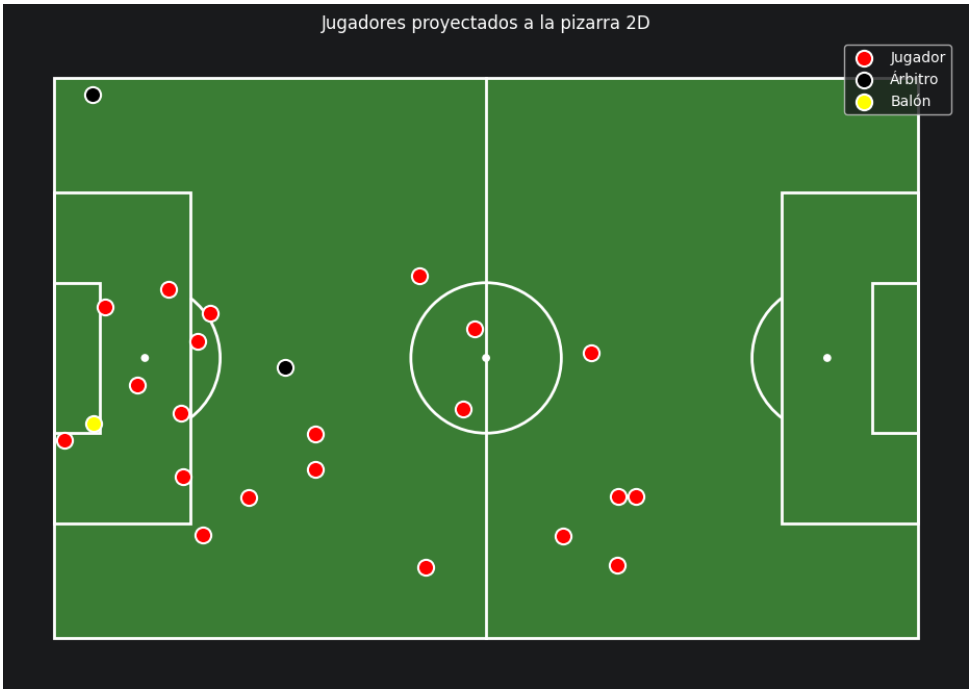


Figura 5.16: Jugadores, árbitros y balón proyectados sobre la pizarra cenital del campo. Fuente: elaboración propia.

5.4 › Motor táctico

Con las posiciones sobre el campo en metros el último módulo ya no hace visión por computador sino lectura del juego. Parte del JSON de la fase anterior y calcula sobre él indicadores colectivos que después pinta sobre la pizarra y sobre el propio vídeo.

5.4.1 › Métricas de bloque

El centroide de un equipo es la posición media de sus jugadores de campo, y seguir su recorrido ya dice mucho, porque si se adelanta es que el equipo ha subido líneas y si retrocede es que se ha replegado cerca de su área. Alrededor de ese centro mido la dispersión, que cuenta cómo de juntos van, la amplitud de lo que ocupan a lo ancho y la profundidad de lo que se estiran en largo. En el clip de prueba el equipo que defendía iba más compacto, con 6,0 m de dispersión frente a los 6,6 m del que llevaba la iniciativa y con amplitudes de 38 y 41 metros (figura 5.17).

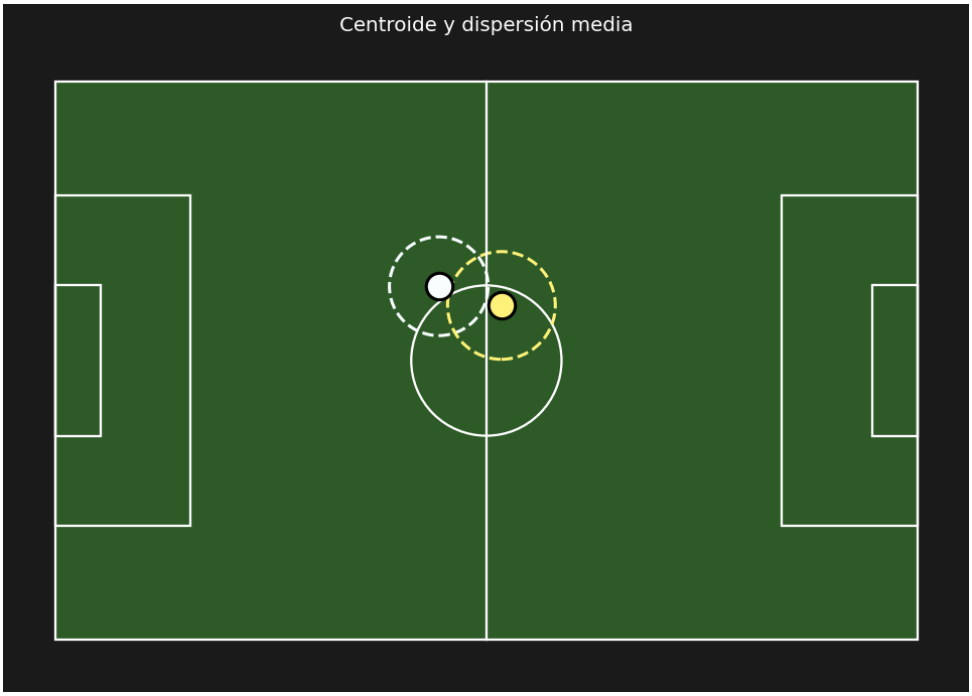


Figura 5.17: Centroide de cada equipo y dispersión de sus jugadores alrededor de él sobre la pizarra. Fuente: elaboración propia.

5.4.2 › Mapas de calor y posesión

El mapa de calor acumula las posiciones de cada equipo sobre una rejilla del campo y la suaviza, con lo que de un vistazo se ve en qué zonas pasa el partido cada conjunto (figura 5.18).

La posesión la reparto por cercanía al balón. En cada fotograma miro qué jugador lo tiene más cerca y se la apunto a su equipo, y cuando nadie está lo bastante próximo ese instante queda como disputa. En el clip de prueba salió un 62 % para el equipo con la iniciativa, un 25 % para el rival y un 13 % de balón sin dueño claro, con su reparto por tercios del campo.

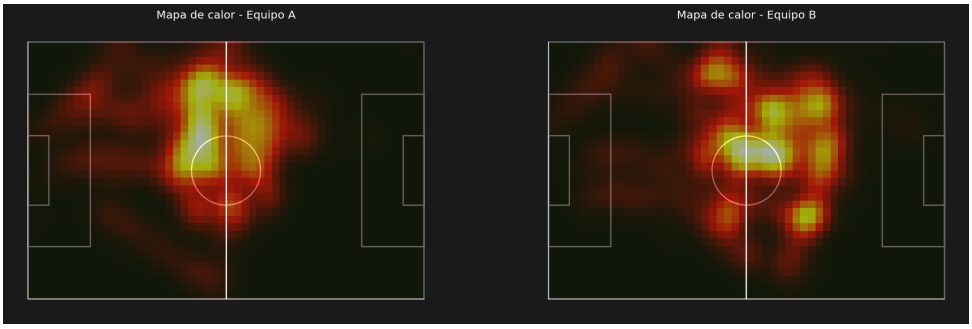


Figura 5.18: Zonas que más ocupa cada equipo a lo largo del partido. Fuente: elaboración propia.

5.4.3 › Formación y dirección de ataque

La dirección de ataque la deduzco del portero. El equipo cuyo guardameta vive pegado a la portería izquierda ataca hacia la derecha, y al revés, sin necesidad de nada más porque el portero es la referencia más fija del campo. En el clip de prueba el equipo local atacaba hacia la derecha.

La formación sale de la posición mediana de cada jugador a lo largo del clip, agrupada en líneas que comparo con un catálogo de esquemas habituales. El sistema leyó un 1-4-2-3-1 (figura 5.19). No siempre cae un dibujo de manual, porque un identificador mal cosido o un jugador que pasa el rato lejos del balón corre una línea de sitio, pero la silueta general del equipo se reconoce.

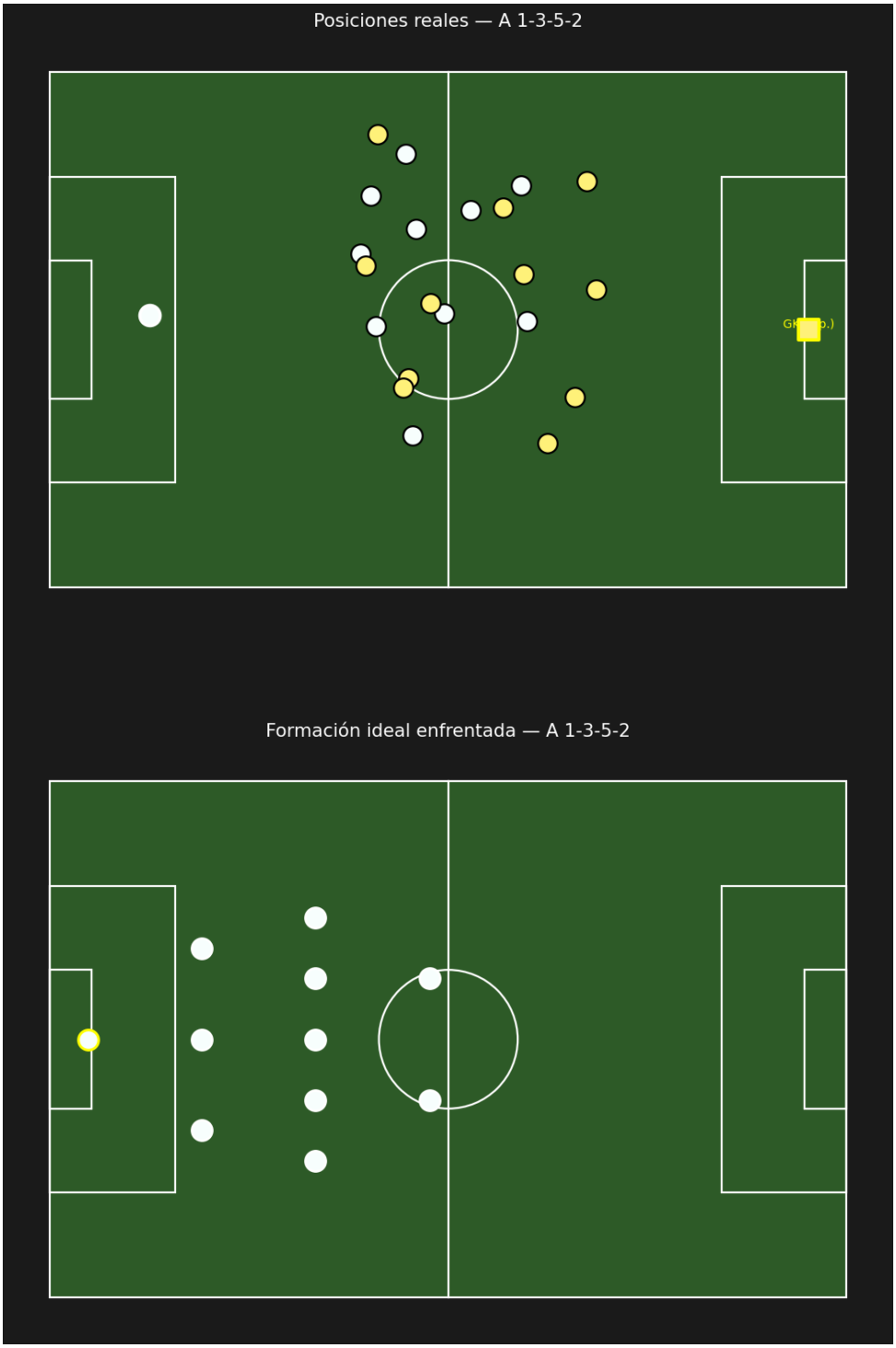


Figura 5.19: Formación leída contra el catálogo, con los jugadores agrupados por líneas y la dirección de ataque inferida. Fuente: elaboración propia.

5.4.4 ❯ Doble visualización

Cada métrica la represento de dos formas a la vez. La pizarra cenital da el plano del campo con los números dibujados encima, cómoda para leer distancias y zonas. La vista en perspectiva proyecta esas mismas marcas sobre el césped del vídeo invirtiendo la homografía de campo a imagen, y ata cada dato a lo que se ve

en pantalla (figura 5.20).



Figura 5.20: La misma jugada sobre la pizarra cenital y proyectada sobre el vídeo en perspectiva. Fuente: elaboración propia.

5.5 › Pipeline integrado

Cada módulo de los apartados anteriores vivía en su propio cuaderno mientras lo desarrollaba. Para procesar un vídeo de principio a fin los reuní en un único notebook que se ejecuta de arriba abajo cambiando una sola línea, la del nombre del archivo de vídeo (figura 5.21).

```
# ============================================================
#  PIPELINE COMPLETO — procesa un vídeo de principio a fin
#  Para cambiar de vídeo: edita SOLO la variable VIDEO_FILE
# ============================================================
import ...

REPO_ROOT = Path("../..").resolve()

# ---- ÚNICO parámetro a cambiar por vídeo ----
VIDEO_FILE = "soccernet_SNMOT-146.mp4"        # <--- cambia esto para otro vídeo
# ---------------------------------------------

VIDEO_IN = REPO_ROOT / "data" / "raw" / VIDEO_FILE
NOMBRE = VIDEO_IN.stem.replace(" ", "_").replace("(", "").replace(")", "")

OUT_NPZ = REPO_ROOT / "data" / "tracking" / f"homografias_pnl_{NOMBRE}.npz"
OUT_JSON = REPO_ROOT / "data" / "tracking" / f"integrado_{NOMBRE}.json"
OUT_NPZ.parent.mkdir(parents=True, exist_ok=True)

# Modelos y dependencias
MODELO_DETECTOR = REPO_ROOT / "models" / "best_v4.pt"
PNL_DIR = REPO_ROOT / "external" / "PnLCalib"

device = "cuda" if torch.cuda.is_available() else "cpu"

# Verificaciones de existencia
assert VIDEO_IN.exists(), f"No existe el vídeo: {VIDEO_IN}"
assert MODELO_DETECTOR.exists(), f"No existe el detector: {MODELO_DETECTOR}"
assert PNL_DIR.exists(), f"No existe PnLCalib: {PNL_DIR}"

cap = cv2.VideoCapture(str(VIDEO_IN))
N_FRAMES = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
W_IMG = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
H_IMG = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
FPS = cap.get(cv2.CAP_PROP_FPS)
cap.release()

print(f"Vídeo: {VIDEO_FILE}")
print(f"  {N_FRAMES} frames, {W_IMG}x{H_IMG}, {FPS:.0f} fps")
print(f"  NOMBRE = {NOMBRE}")
print(f"  device = {device}")
print(f"Salidas: {OUT_NPZ.name}, {OUT_JSON.name}")
[1]
```

Figura 5.21: Cabecera del notebook integrado, donde el único parámetro que se cambia entre vídeos es `VIDEO_FILE`. Fuente: elaboración propia.

A partir de ese nombre el cuaderno encadena toda la tubería sin más intervención. Calibra el vídeo con PnLCalib y guarda las homografías, descartando los pocos fotogramas cuya proyección se dispara por encima de los 120 metros. Detecta y sigue a los jugadores con BoT-SORT, cose los tracklets partidos y deja una identidad por persona, lo que en el clip de ejemplo SNMOT-146, distinto del clip de validación usado en los apartados anteriores, baja el recuento de 157 identidades en crudo a 33. Después asigna el equipo por color y resuelve el rol de cada uno.

La salida es el JSON integrado que ya detallé en la metodología, con una entrada por objeto y fotograma. El siguiente fragmento muestra su forma sobre uno de los

clips procesados.

```
{
  "video": "soccernet_SNMOT-146.mp4",
  "n_frames": 750,
  "color_equipo_bgr": {"0": [246, 254, 253], "1": [90, 175, 254]},
  "frames": [
    [
      {"id": 2,  "rol": "jugador", "equipo": 0,  "bbox": [812,430,845,510], "campo"
      {"id": 61, "rol": "portero", "equipo": 0,  "bbox": [120,360,158,470], "campo"
      {"id": 14, "rol": "arbitro", "equipo": -1, "bbox": [980,300,1010,380], "campo
    ]
  ]
}
```

La arquitectura del flujo está en la figura 4.2 de ese capítulo, así que aquí basta con recordar que cada módulo entrega su resultado al siguiente y que ese JSON es la interfaz que los une.

5.6 › Evaluación cuantitativa

La prueba definitiva no es un clip escogido sino el conjunto de test al completo. Procesé los 49 clips de SoccerNet-Tracking con una versión reducida del pipeline, sin la calibración de PnLCalib, que es demasiado lenta para tantas secuencias y aquí no hace falta, porque todas las métricas viven en el plano de la imagen y comparan las cajas del sistema con la verdad de referencia fotograma a fotograma.

5.6.1 › Detección, seguimiento y clasificación

La detección de personas se sostiene a lo largo de todo el test, con una F1 de 0,926. El árbitro queda algo por debajo, en 0,880, y el balón vuelve a ser el punto flaco, con una F1 de 0,564 que arrastra su baja exhaustividad, porque se detecta en torno a la mitad de los fotogramas en los que aparece. La tabla 5.4 recoge las tres clases.

Clase	Precisión	Exhaustividad	F1
Persona	0,941	0,910	0,926
Árbitro	0,872	0,887	0,880
Balón	0,790	0,438	0,564

Tabla 5.4: Detección por clase sobre los 49 clips de test (micro-promedio). Fuente: elaboración propia.

El seguimiento, acumulado sobre los 49 clips, da una MOTA de 0,847 y una IDF1 de 0,678. La MOTA sale alta porque depende sobre todo de la detección, que es buena. La IDF1 baja más, por los cambios de identidad que dejan las oclusiones aun después del cosido.

La clasificación de equipos por color acierta en el 91,7 % de las detecciones emparejadas, con una media por clip del 92,1 % y un mínimo del 78,8 % en la secuencia más difícil.

5.6.2 › Efecto del post-procesado

Donde más se nota el post-procesado es en el balón. Tal y como sale del detector, su trayectoria solo cubre el 51,5 % de los fotogramas. Al interpolar los huecos cortos esa cobertura sube hasta el 82 % y la F1 mejora de 0,564 a 0,581. Se paga con precisión, que cae de 0,790 a 0,619 porque rellenar inventa alguna posición donde el balón no estaba, pero la trayectoria queda mucho más aprovechable para el análisis posterior. La tabla 5.5 enfrenta las dos versiones.

Balón	Precisión	Exhaustividad	F1	Cobertura
Crudo	0,790	0,438	0,564	51,5 %
Interpolado	0,619	0,547	0,581	82,0 %

Tabla 5.5: Efecto de la interpolación de huecos cortos en la trayectoria del balón, sobre los 49 clips. Fuente: elaboración propia.

CAPÍTULO 6

Conclusiones

El trabajo termina con un sistema que hace lo que se propuso. A partir de un único vídeo de retransmisión y sobre una máquina de consumo el pipeline detecta a los jugadores, al árbitro y al balón, sostiene su identidad a lo largo de la secuencia, reparte a cada futbolista en su equipo, calibra el campo, proyecta las posiciones sobre un plano en dos dimensiones y calcula sobre ellas un conjunto de métricas tácticas. El recorrido completo del fichero de vídeo a la pizarra queda cubierto de principio a fin, que era el objetivo general del que partía todo lo demás.

Mirando los objetivos específicos uno a uno, el balance es desigual pero en conjunto favorable. La detección de personas es la pieza más sólida con una F1 que ronda el 0,93 sobre el conjunto de test y que se mantiene estable entre partidos y equipaciones distintas. El seguimiento conserva la identidad con una MOTA alta, arrastrada sobre todo por la calidad de la detección, mientras que la IDF1, que es la que de verdad importa para reconstruir trayectorias, se queda más abajo por los cambios de identidad que dejan las oclusiones aun después del cosido. La clasificación de equipos por color acierta en torno al noventa y dos por ciento de las detecciones emparejadas, una cifra que para un método sin etiquetas previas resulta más que razonable.

El punto flaco está donde se esperaba que estuviera. El balón es pequeño, va rápido y desaparece detrás de los jugadores cada pocos fotogramas, y eso se traduce en una exhaustividad baja que ningún ajuste del detector ha conseguido enderezar del todo. La interpolación de los huecos cortos alivia el problema y sube la cobertura de su trayectoria de la mitad a más de cuatro quintos de los fotogramas, a costa de

inventar alguna posición, pero el balón sigue siendo el objeto que peor se comporta y el primero al que habría que dedicarse en una continuación del trabajo.

La calibración merece un comentario aparte porque su historia resume bien el carácter iterativo del proyecto. La idea de partida era resolverla con un modelo propio de keypoints del campo, y se llegó a entrenar y a afinar. El problema no estuvo en la precisión del modelo en sí, sino en que demasiados fotogramas no mostraban suficientes puntos visibles para fijar una homografía estable, algo inherente a un plano que recorta el campo y enseña pocas marcas a la vez. Esa carencia fue la que obligó a apoyarse en PnLCalib, un método de terceros que cubre el vídeo entero con más holgura. La decisión no estaba en el plan inicial y es un buen ejemplo de cómo la validación al cierre de cada fase empujó a rehacer una parte que sobre el papel parecía resuelta.

Sobre esa calibración se levanta la proyección y ahí el resultado se juzga mirando la pizarra más que una cifra. Cuando la homografía es fiable los jugadores caen dentro de los límites del campo, los dos equipos se reparten en lados coherentes y la trayectoria del balón se sigue sin sobresaltos. Que la evaluación de esta parte sea cualitativa y no numérica responde a una razón de método. PnLCalib ya venía contrastado por sus autores, y volver a medirlo sobre datos del mismo tipo habría puntuado el método de un tercero sin aislar lo que de verdad aporta el trabajo que es engancharlo con la detección, el seguimiento y el motor táctico dentro de una misma tubería. Por eso lo que se valora es su efecto sobre la representación final y no su error por separado.

El motor táctico cumple su cometido. Sobre las posiciones proyectadas obtiene el centroide, la amplitud del bloque, los mapas de calor, la posesión y la orientación del ataque y los representa tanto sobre la pizarra como sobre la propia imagen. Son medidas que tienen lectura para quien entiende de juego y que dejan claro que, una vez se dispone del dato posicional, el salto a la interpretación táctica es corto. La integración de todo ello en una sola tubería que procesa un vídeo cambiando un único parámetro era otro de los objetivos, y también queda cubierto, igual que la publicación en abierto de los modelos y el código.

Por debajo de esos resultados hay una manera de trabajar que también deja poso. El proyecto avanzó como un ciclo en el que cada fase se cerraba validando sobre un vídeo aparte, y ese hábito obligó a deshacer más de una decisión por el camino. La comparación entre ByteTrack y BoT-SORT, la lectura de dorsales por reconocimiento óptico que no llegó a cuajar, la corrección cinemática de los intercambios de identidad o el auto-etiquetado del detector se quedaron por el camino, cada uno con su motivo apuntado. Todos empujaron el sistema hacia la forma que acabó teniendo, y reconocerlos evita vender como lineal un desarrollo que no lo fue en absoluto.

Hay además un condicionante que ha estado presente en casi todas las decisiones y que conviene reconocer al cerrar. Los seis gigabytes de memoria de la tarjeta gráfica marcaron desde el principio qué modelos cabían y a qué resolución podían ejecutarse. Trabajar la detección a 1280 píxeles para no perder a los jugadores lejanos ni al balón, dejar fuera las alternativas más pesadas o reservar la calibración costosa para tramos concretos no fueron caprichos sino consecuencias de ese techo. Que el sistema entero se sostenga sobre una máquina de consumo no es un efecto secundario, es justo lo que se quería demostrar y es lo que separa esta propuesta de las que solo respiran en un servidor con gráficas de sobra.

Queda, por tanto, la sensación de un objetivo general alcanzado con honestidad sobre sus límites. El sistema no compite con una instalación multicámara ni lo pretende. Lo que demuestra es que desde la fuente más accesible que existe y con un presupuesto de cómputo modesto se puede recuperar un poco del análisis posicional que hasta hace poco exigía infraestructura cara, y que esa porción basta para describir el juego en términos tácticos. Ahí está el valor del trabajo, más en haber sostenido la cadena entera de extremo a extremo que en cualquiera de sus piezas por separado.

6.1 › Líneas futuras de trabajo

La continuación más evidente pasa por el balón. Un detector específico para objetos pequeños, entrenado con más ejemplos de balón en situaciones difíciles, ganaría la exhaustividad que hoy falta. A eso se le puede sumar un modelo de trayectoria que aproveche la física del movimiento para sostener la pelota cuando queda tapada en lugar de limitarse a interpolar los huecos a posteriori. Es la mejora que más se notaría en la calidad del dato final.

La calibración deja también un frente abierto. Reunir y anotar más fotogramas de campo permitiría retomar el modelo propio de keypoints y, si llegara a fijar homografías estables en una proporción suficiente de imágenes, prescindir de la dependencia de PnLCalib o reservarla solo para los tramos complicados. Por ese mismo camino está la cuestión del coste de cómputo ya que la calibración es la parte más lenta de la tubería y hoy obliga a renunciar a ella cuando se procesan muchas secuencias seguidas.

Tampoco vendría mal ampliar y diversificar los datos. El detector se entrenó sobre todo con imágenes de una misma competición, y aunque la evaluación sobre SoccerNet enseña que aguanta en escenarios distintos, sumar partidos de más ligas, con cámaras y realizaciones diferentes, lo volvería más robusto. La calibración agradecería lo mismo, fotogramas de estadios y encuadres que hoy no están representados y que de paso ayudarían a recuperar a medio plazo el modelo propio de keypoints.

En el seguimiento el que más castigo recibe son los cambios de identidad. Incorporar la lectura del dorsal mediante reconocimiento óptico de caracteres, o descriptores de apariencia más ricos para reidentificar a un jugador tras una oclusión larga, recortaría esos cortes que hoy parten en dos la trayectoria de un mismo futbolista. Cualquier avance en esa dirección repercute directamente en la IDF1, que es la métrica que de verdad mide la utilidad del seguimiento para el análisis.

Hay margen igualmente en lo táctico. Las posiciones proyectadas abren la puerta a métricas más elaboradas que las implementadas como el control de espacio mediante diagramas de Voronoi, la amenaza esperada de cada posesión o la detección automática de pases y eventos. Todas ellas se apoyan en el mismo dato posicional que el sistema ya produce de modo que el esfuerzo se concentraría en la capa de interpretación y no en la de percepción.

Más allá de cada módulo quedan dos caminos de mayor alcance. Uno es la optimización hacia un funcionamiento en tiempo real con modelos más ligeros y un procesamiento por lotes que acerque el sistema a un uso en directo. El otro es el de la herramienta, envolver el pipeline en una interfaz que permita a un entrenador cargar un vídeo y recibir las visualizaciones sin tocar el código, y validar con usuarios reales hasta qué punto lo que el sistema mide les resulta de ayuda. Esa prueba, la del valor para quien analiza el juego, es la que de verdad cerraría el círculo que la introducción dejó abierto.

CAPÍTULO 7

Abreviaturas

A lo largo de la memoria aparecen acrónimos y términos técnicos que, salvo en su primera aparición, se emplean en su forma abreviada. La siguiente lista recoge su significado para facilitar la lectura.

Abreviatura	**Significado**
AssA	Association Accuracy, exactitud de asociación en seguimiento
BoT-SORT	Seguidor multiobjeto que combina movimiento y reidentificación sobre la base de SORT
ByteTrack	Algoritmo de seguimiento multiobjeto que asocia también las detecciones de baja confianza
CIELAB	Espacio de color L*a*b* definido por la Commission Internationale de l'Éclairage
CIoU	Complete Intersection over Union
CNN	Convolutional Neural Network, red neuronal convolucional
CRISP-DM	Cross-Industry Standard Process for Data Mining
CUDA	Compute Unified Device Architecture
CVAT	Computer Vision Annotation Tool
DeepPose	Estimación de pose humana por regresión directa con redes profundas
DeepSORT	Extensión de SORT con descriptores de apariencia
DetA	Detection Accuracy, componente de detección de HOTA
DETR	DEtection TRansformer
DFL	Deutsche Fußball Liga

Abreviatura	Significado
DLT	Direct Linear Transform, transformación lineal directa
EPTS	Electronic Performance and Tracking Systems
F1	Media armónica de la precisión y la exhaustividad
GPS	Global Positioning System
GPU	Graphics Processing Unit, unidad de procesamiento gráfico
GSR	Game State Reconstruction, reconstrucción del estado de juego
GS-HOTA	Variante de HOTA adaptada a la reconstrucción del estado de juego
HOG	Histogram of Oriented Gradients
HOTA	Higher Order Tracking Accuracy
HRNet	High-Resolution Network
HSV	Hue, Saturation, Value
IdSim	Identity Similarity, similitud de identidad
IDF1	Identity F1, métrica de consistencia de identidad en seguimiento
IoU	Intersection over Union, intersección sobre la unión
JSON	JavaScript Object Notation
K-means	Algoritmo de agrupamiento por k-medias
LocSim	Localization Similarity, similitud de localización
mAP	mean Average Precision, precisión media
MIT	Licencia de software permisiva del Massachusetts Institute of Technology
MOT	Multiple Object Tracking
MOTA	Multiple Object Tracking Accuracy
OCR	Optical Character Recognition, reconocimiento óptico de caracteres
OpenCV	Open Source Computer Vision Library
OpenPose	Estimación de pose multipersona en tiempo real
PnLCalib	Método de calibración de campos deportivos basado en puntos y líneas
RAM	Random Access Memory
RANSAC	Random Sample Consensus
RGB	Red, Green, Blue
SIFT	Scale-Invariant Feature Transform
SORT	Simple Online and Realtime Tracking
TFG	Trabajo de Fin de Grado
ViTPose	Modelo de estimación de pose basado en Vision Transformer
YOLO	You Only Look Once
YOLOv8	Versión 8 de la familia de detectores YOLO

Abreviatura	Significado
YOLO-pose	Variante de YOLO para la estimación de keypoints

Índice de figuras

4.1. Fases de CRISP-DM y su correspondencia con las del proyecto 33
4.2. Arquitectura del sistema 36

5.1. Pipeline de aumento fotométrico (notebook 11) 41
5.2. Distribución de clases y tamaño de las cajas 42
5.3. Configuración y lanzamiento del entrenamiento 43
5.4. Curvas de entrenamiento y validación del detector v5 43
5.5. Matriz de confusión del detector final 44
5.6. Curva precisión–recall del detector v5 45
5.7. Configuración de BoT-SORT 46
5.8. Seguimiento con identificadores 47
5.9. División por clase y cosido de tracklets 48
5.10. Color de camiseta por identidad 49
5.11. Muestra de validación de la clasificación 50
5.12. Keypoints detectados por el modelo base 51
5.13. Exclusión de keypoints poco fiables 52
5.14. Reproyección con y sin los keypoints poco fiables 53
5.15. Primer enfoque: propagación de la homografía por movimiento de cámara 53
5.16. Pizarra 2D con las detecciones proyectadas 55
5.17. Centroide y dispersión del bloque 56
5.18. Mapa de calor de ocupación 56
5.19. Formación detectada y dirección de ataque 58
5.20. Doble visualización: cenital y en perspectiva 59
5.21. Parámetro único del pipeline 60

Índice de tablas

4.1. Pila de software y versiones empleadas. 35

5.1. Comparativa de los detectores v1–v4 44
5.2. Resultados del detector v5 45
5.3. Fine-tuning de keypoints: validación frente a test 52
5.4. Detección sobre el conjunto de test 61
5.5. Trayectoria del balón: crudo frente a interpolado 62

Bibliografía

[1] R. Rein y D. Memmert, «Big data and tactical analysis in elite soccer: future challenges and opportunities for sports science,» *SpringerPlus*, vol. 5, n.º 1, pág. 1410, 2016. DOI: 10.1186/s40064-016-3108-2 dirección: https://doi.org/10.1186/s40064-016-3108-2

[2] A. Ferraz, P. Duarte-Mendes, H. Sarmento, J. Valente-Dos-Santos y B. Travassos, «Tracking devices and physical performance analysis in team sports: a comprehensive framework for research—trends and future directions,» *Frontiers in Sports and Active Living*, vol. 5, pág. 1 284 086, 2023. DOI: 10.3389/fspor.2023.1284086 dirección: https://doi.org/10.3389/fspor.2023.1284086

[3] B. T. Naik, M. F. Hashmi y N. D. Bokde, «A Comprehensive Review of Computer Vision in Sports: Open Issues, Future Trends and Research Directions,» *Applied Sciences*, vol. 12, n.º 9, pág. 4429, 2022. DOI: 10.3390/app12094429 dirección: https://doi.org/10.3390/app12094429

[4] A. Cioppa et al., «SoccerNet-Tracking: Multiple Object Tracking Dataset and Benchmark in Soccer Videos,» en *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2022. dirección: https://arxiv.org/abs/2204.06918

[5] M. Rico-González, J. Pino-Ortega, F. Y. Nakamura, F. A. Moura y A. Los Arcos, «Identification, Computational Examination, Critical Assessment and Future Considerations of Spatial Tactical Variables to Assess the Use of Space in Team Sports by Positional Data: A Systematic Review,» *Journal of Human Kinetics*, vol. 77, págs. 205-221, 2021. DOI: 10.2478/hukin-2021-0021 dirección: https://doi.org/10.2478/hukin-2021-0021

[6] M. Rico-González, J. Pino-Ortega, F. Y. Nakamura, F. A. Moura y A. Los Arcos, «Identification, Computational Examination, Critical Assessment and Future Considerations of Distance Variables to Assess Collective Tactical Behaviour in Team Invasion Sports by Positional Data: A Systematic Review,» *International Journal of Environmental Research and Public Health*, vol. 17, n.º 6, pág. 1952, 2020. DOI: 10.3390/ijerph17061952 dirección: https://doi.org/10.3390/ijerph17061952

[7] Z. Wang, P. Veličković, D. Hennes, N. Tomašev et al., «TacticAI: an AI assistant for football tactics,» *Nature Communications*, vol. 15, pág. 1906, 2024. DOI: 10.1038/s41467-024-45965-x dirección: https://doi.org/10.1038/s41467-024-45965-x

[8] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2.ª ed. Springer, 2022. dirección: https://szeliski.org/Book/

[9] N. Dalal y B. Triggs, «Histograms of Oriented Gradients for Human Detection,» en *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2005, págs. 886-893. DOI: 10.1109/CVPR.2005.177 dirección: https://inria.hal.science/inria-00548512

[10] A. Krizhevsky, I. Sutskever y G. E. Hinton, «ImageNet Classification with Deep Convolutional Neural Networks,» en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012. dirección: https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

[11] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. dirección: https://arxiv.org/abs/1706.03762

[12] A. Dosovitskiy et al., «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,» en *International Conference on Learning Representations (ICLR)*, 2021. dirección: https://arxiv.org/abs/2010.11929

[13] S. Ren, K. He, R. Girshick y J. Sun, «Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,» en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015. dirección: https://arxiv.org/abs/1506.01497

[14] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, «You Only Look Once: Unified, Real-Time Object Detection,» en *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, págs. 779-788. DOI: 10.1109/CVPR.2016.91 dirección: https://arxiv.org/abs/1506.02640

[15] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov y S. Zagoruyko, «End-to-End Object Detection with Transformers,» en *Computer Vision – ECCV 2020*, Springer, 2020, págs. 213-229. dirección: https://arxiv.org/abs/2005.12872

[16] J. Terven, D.-M. Córdova-Esparza y J.-A. Romero-González, «A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS,» *Machine Learning and Knowledge Extraction*, vol. 5, n.º 4, págs. 1680-1716, 2023. DOI: 10.3390/make5040083 dirección: https://doi.org/10.3390/make5040083

[17] R. E. Kalman, «A New Approach to Linear Filtering and Prediction Problems,» *Journal of Basic Engineering*, vol. 82, n.º 1, págs. 35-45, 1960. DOI: 10.1115/1.3662552 dirección: https://doi.org/10.1115/1.3662552

[18] H. W. Kuhn, «The Hungarian Method for the Assignment Problem,» *Naval Research Logistics Quarterly*, vol. 2, n.º 1-2, págs. 83-97, 1955. DOI: 10.1002/nav.3800020109 dirección: https://doi.org/10.1002/nav.3800020109

[19] A. Bewley, Z. Ge, L. Ott, F. Ramos y B. Upcroft, «Simple Online and Realtime Tracking,» en *2016 IEEE International Conference on Image Processing (ICIP)*, 2016, págs. 3464-3468. DOI: 10.1109/ICIP.2016.7533003 dirección: https://arxiv.org/abs/1602.00763

[20] N. Wojke, A. Bewley y D. Paulus, «Simple Online and Realtime Tracking with a Deep Association Metric,» en *2017 IEEE International Conference on Image Processing (ICIP)*, 2017, págs. 3645-3649. DOI: 10.1109/ICIP.2017.8296962 dirección: https://arxiv.org/abs/1703.07402

[21] Y. Zhang et al., «ByteTrack: Multi-Object Tracking by Associating Every Detection Box,» en *Computer Vision – ECCV 2022*, Springer, 2022. dirección: https://arxiv.org/abs/2110.06864

[22] N. Aharon, R. Orfaig y B.-Z. Bobrovsky, «BoT-SORT: Robust Associations Multi-Pedestrian Tracking,» *arXiv preprint arXiv:2206.14651*, 2022. dirección: https://arxiv.org/abs/2206.14651

[23] Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu y L. Wang, «SportsMOT: A Large Multi-Object Tracking Dataset in Multiple Sports Scenes,» en *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. dirección: https://arxiv.org/abs/2304.05170

[24] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang e Y. Wei, «MOTR: End-to-End Multiple-Object Tracking with Transformer,» en *Computer Vision – ECCV 2022*, Springer, 2022. dirección: https://arxiv.org/abs/2105.03247

[25] J. MacQueen, «Some Methods for Classification and Analysis of Multivariate Observations,» en *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, University of California Press, 1967, págs. 281-297. dirección: https://projecteuclid.org/euclid.bsmsp/1200512992

[26] D. Arthur y S. Vassilvitskii, «k-means++: The Advantages of Careful Seeding,» en *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, SIAM, 2007, págs. 1027-1035. dirección: https://dl.acm.org/doi/10.5555/1283383.1283494

[27] A. Toshev y C. Szegedy, «DeepPose: Human Pose Estimation via Deep Neural Networks,» en *2014 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, págs. 1653-1660. DOI: 10.1109/CVPR.2014.214 dirección: https://arxiv.org/abs/1312.4659

[28] A. Newell, K. Yang y J. Deng, «Stacked Hourglass Networks for Human Pose Estimation,» en *Computer Vision – ECCV 2016*, Springer, 2016, págs. 483-499. dirección: https://arxiv.org/abs/1603.06937

[29] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei e Y. Sheikh, «OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,» *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, n.º 1, págs. 172-186, 2021. DOI: 10.1109/TPAMI.2019.2929257 dirección: https://arxiv.org/abs/1812.08008

[30] K. Sun, B. Xiao, D. Liu y J. Wang, «Deep High-Resolution Representation Learning for Human Pose Estimation,» en *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, págs. 5693-5703. DOI: 10.1109/CVPR.2019.00584 dirección: https://arxiv.org/abs/1902.09212

[31] Y. Xu, J. Zhang, Q. Zhang y D. Tao, «ViTPose: Simple Vision Transformer Baselines for Human Pose Estimation,» en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, págs. 38 571-38 584. dirección: https://arxiv.org/abs/2204.12484

[32] D. Maji, S. Nagori, M. Mathew y D. Poddar, «YOLO-Pose: Enhancing YOLO for Multi-Person Pose Estimation Using Object Keypoint Similarity Loss,» en *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2022, págs. 2637-2646. DOI: 10.1109/CVPRW56347.2022.00297 dirección: https://arxiv.org/abs/2204.06806

[33] A. Cioppa, A. Deliège, S. Giancola, B. Ghanem y M. Van Droogenbroeck, «Scaling up SoccerNet with Multi-View Spatial Localization and Re-Identification,» *Scientific Data*, vol. 9, n.º 1, pág. 355, 2022. DOI: 10.1038/s41597-022-01469-1 dirección: https://doi.org/10.1038/s41597-022-01469-1

[34] M. Gutiérrez-Pérez y A. Agudo, «PnLCalib: Sports Field Registration via Points and Lines Optimization,» *arXiv preprint arXiv:2404.08401*, 2024. dirección: https://arxiv.org/abs/2404.08401

[35] R. Hartley y A. Zisserman, *Multiple View Geometry in Computer Vision*, 2.ª ed. Cambridge University Press, 2004. dirección: https://www.robots.ox.ac.uk/~vgg/hzbook/

[36] M. A. Fischler y R. C. Bolles, «Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,» *Communications of the ACM*, vol. 24, n.º 6, págs. 381-395, 1981. DOI: 10.1145/358669.358692 dirección: https://doi.org/10.1145/358669.358692

[37] V. Somers et al., «SoccerNet Game State Reconstruction: End-to-End Athlete Tracking and Identification on a Minimap,» en *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2024. dirección: https://arxiv.org/abs/2404.11335

[38] J. Luiten et al., «HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,» *International Journal of Computer Vision*, vol. 129, n.º 2, págs. 548-578, 2021. DOI: 10.1007/s11263-020-01375-2 dirección: https://doi.org/10.1007/s11263-020-01375-2

[39] Metrica Sports, *Metrica Sports: video analysis and tracking software*, 2024. visitado 13 de jun. de 2026. dirección: https://www.metrica-sports.com/

[40] R. Wirth y J. Hipp, «CRISP-DM: Towards a Standard Process Model for Data Mining,» en *Proc. 4th Int. Conf. Practical Application of Knowledge Discovery and Data Mining (PADD)*, Manchester, UK, 2000, págs. 29-39.

[41] A. Paszke et al., «PyTorch: An Imperative Style, High-Performance Deep Learning Library,» en *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, págs. 8024-8035.

[42] G. Jocher, A. Chaurasia y J. Qiu, *Ultralytics YOLOv8*, ver. 8.0.0, Licencia AGPL-3.0, 2023. dirección: https://github.com/ultralytics/ultralytics

[43] Deutsche Fußball Liga, *DFL – Bundesliga Data Shootout (Kaggle competition)*, 2022. visitado 24 de jun. de 2026. dirección: https://www.kaggle.com/competitions/dfl-bundesliga-data-shootout

[44] K. Bernardin y R. Stiefelhagen, «Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,» *EURASIP Journal on Image and Video Processing*, vol. 2008, pág. 246 309, 2008. DOI: 10.1155/2008/246309 dirección: https://doi.org/10.1155/2008/246309

[45] E. Ristani, F. Solera, R. Zou, R. Cucchiara y C. Tomasi, «Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking,» en *Computer Vision – ECCV 2016 Workshops*, Springer, 2016, págs. 17-35. dirección: https://arxiv.org/abs/1609.01775